00　ECS × EFS × Log4j2「Stale file handle」完全解説

java.io.IOException: Stale file handle の正体・原因の順位・確かめ方・切り分け・緊急対応／恒久対策／運用回避を、仕組みと歴史から
2026年10月時点（AWS 公式ドキュメント・Linux カーネル／NFS の一次情報・Docker での実機検証で確認）

■ ご質問への答え（要約）

質問	答え
Q1　何が起きているのか（一言で）	Stale file handle（ステイル・ファイル・ハンドル＝古くなった引換券）は、アプリ（Log4j2）が握っている「ファイルの引換券」の品物が、EFS の上でもう消された・作り直されたときに出るエラー。Java の例外は Linux のエラー番号 ESTALE（116）をそのまま文字にしたもの。アプリ本体は動き続けるが、そのあいだのログは失われる。
Q2　仮説「デプロイ後に EFS のマウントを外して付け直した」は成り立つか	そのままの形では成り立たない（可能性はごく低い）。ECS は動いているタスクの EFS を付け外ししない（Fargate ではコンテナの中からも外せない）。仮に付け直しても、開いているファイルは外したマウント経由で書き続けるので ESTALE にはならない（実機 S5：エラー 0・全行保存）。ただし「デプロイの前後で起きた」という観察は大事な手がかり。デプロイ中は新旧タスクが同じログファイルを同時に開くので、下の 1 位の原因の引き金になる。「付け外し」を「EFS 上のファイル・ディレクトリを消した／作り直した」と読み替えれば成り立つ。
Q3　いちばん可能性が高い原因は	1 位：複数のタスクが EFS 上の同じログファイル（/mnt/logs/front/inter-api/logs/…）に書いていて、どれかのタスクの Log4j2 がロールオーバ（改名→gz 圧縮→元ファイル削除、Delete）で、他のタスクが開いているファイルを消した。2 位：ログの片付け（日数・名前で消すバッチ）が動いているタスクのファイルを消した。3 位：デプロイ手順・起動スクリプト・手作業でディレクトリを消して作り直した。4 位：マウントの根元（rootDirectory・アクセスポイントのルート）を作り直した（このときは ls /mnt/logs 自体が Stale）。全 9 項目は 06_原因一覧。
Q4　今すぐやることは	① CloudWatch Logs で Stale を出しているタスク・コンテナと時刻を洗い出す　② ECS Exec でコンテナに入り、根元が Stale か（→ タスク入れ替えしかない）、ファイルだけか（→ JBoss の reload かタスク入れ替え）を確かめる　③ 消している側（他タスクのロールオーバ・片付け・手作業）を止める　④ 影響タスクを 1 つずつ入れ替える（共有パスのままだと入れ替えの重なりで再発しうるので、ロールオーバの時刻を避ける）。手順は 09_緊急対応。
Q5　根本対策は	「1 つのファイルを書くのは 1 タスクだけ」にする。アプリログのリンク先をタスクごとのディレクトリにする（JBoss ログと同じ二段リンク。アプリの変更なし。impl/ に部品）。EFS 上の削除は「止まったタスクのものだけ」（tools/efs-log-housekeeping.sh）。ログの正本は CloudWatch Logs。実機 S7 で Stale 0 件・欠けた行 0 を確認。
Q6　運用で避けるには	EFS 上で「動いているタスクのファイル・ディレクトリを消す・移す・作り直す」をしない。片付けは止まったタスクのものだけ、デプロイ中はしない。構成を変えるときは「新しいパスへ切り替えてから、古い方を最後に消す」。禁止操作とチェックリストは 11_運用回避。

■ 目次（シート名をクリックすると移動）

シート	内容
01_結論	答えの全体像・仮説の評価・原因の順位・対処の早見表

02_たとえ話	小学生にもわかる「引換券とロッカー」「家の机と学校の倉庫」のたとえ
03_歴史と背景	NFS・ESTALE・EFS・コンテナのログが生まれた順番と、この設計が必要になった経緯
04_動作原理	ファイルハンドル・削除の扱い・ECS と EFS のつながり方・Log4j2・エラーの伝わり方
05_動作イメージ	6 つの場面を「時刻 × 登場人物」の表で 1 コマずつ追う
06_原因一覧	可能性の高い順に 9 項目。仕組み・見分け方・緊急／恒久／運用の対処まで
07_確認方法	事象の確かめ方（CloudWatch Logs・スタックトレース・コンテナの中・ECS・EFS）
08_切り分け	はい／いいえで原因を絞り込む手順と、時刻のパターンでの見分け方
09_緊急対応	今すぐ直す手順（範囲 → 証拠 → 止める → 直す → 確認）
10_恒久対策	根本対策（1 ファイル 1 書き手・安全な片付け・根元を守る・正本は CloudWatch Logs）
11_運用回避	EFS 上の禁止操作・片付けのルール・変更手順・チェックリスト
12_コマンド集	調査・復旧・検証に使うコマンドを 1 か所に
13_実装	同梱の診断・片付け・起動スクリプト・Log4j2 設定・Terraform とテスト
14_検証結果	Docker で NFS を立てて再現した結果（実機検証 14 ケース中 14 ケースが想定どおり）
15_用語集	用語を小学生向けのたとえつきで
16_参考資料	根拠にした公式ドキュメント・一次情報

■ 凡例

表記	意味
★★★★★ ～ ☆☆☆☆☆	この環境（ECS + EFS の共有パス + Log4j2）で原因である見込み（筆者の評価）。★ が多いほど先に疑う
ESTALE / Stale file handle	Linux のエラー番号 116。「その引換券（ファイルハンドル）の品物はもう無い」
タスク A ／ タスク B	ECS のタスク（それぞれが別の NFS クライアント）。A = 書いている側、B = 消してしまう側
NFSv4.1 ／ NFSv3	NFS の版。EFS は v4.1（既定）。検証では比較のため v3 も使った
S1 ～ S10	14_検証結果の実験番号（Docker で NFS サーバとクライアント 2 台を立てて再現）
根元	マウントのいちばん上のディレクトリ（タスク定義の rootDirectory、またはアクセスポイントのルート）
いまの構成	/webapp/webapp9mf02/logs → /mnt/logs/front/inter-api/logs（同じサービスの全タスクが共有）

作成: 2026-10-09 ／ フォント: Meiryo UI ／ 配色: モノトーン ／ 本文は Markdown 版と同じ

01　結論：何が起きていて、どうすればよいか

Stale file handle は「アプリが握っている引換券（ファイルハンドル）の品物が、EFS の上でもう無い」という意味。原因はほぼ必ず「誰か（別のタスク・別のホスト・作業者）が、そのファイルやディレクトリを消した・作り直した」こと。マウントの付け外しそのものは原因にならない。

■ 何が起きたのか（全体像）



・シンボリックリンク（/webapp/webapp9mf02/logs → /mnt/logs/…）は原因ではない。リンクは「開くとき 1 回」たどるだけで、開いたあとの書き込みは実体（引換券）に直接向かう。

・ローカルディスクなら「誰かが消しても、開いている人が閉じるまで中身は残る」ので書き込みは成功する（見えなくなるだけ）。EFS（NFS）では、別のクライアントが消すと、開いている側の引換券はその場で無効になる。ここがいちばん大事な違い。

・Log4j2 は書けなくなっても自動で開き直さない。次のロールオーバかアプリの再起動まで、ログは失われ続ける。

・コンテナの OS（UBI 9.8）・OpenJDK 21・JBoss EAP 8.1 の版は原因ではない。NFS のやりとりはホスト（Fargate の VM・EC2）のカーネルが行う。

■ 仮説「アプリをデプロイした後で、EFS のマウントを外して、再度アタッチした」の評価

読み方（何をしたか）	成り立つか	理由（根拠）
動いているタスクの EFS ボリュームを外して付け直した	成り立たない	ECS（EC2・Fargate とも）は動いているタスクのボリュームを付け外ししない。タスク定義を変えて再デプロイすると「新しいタスク」が「新しく」マウントする（引換券も新しいので Stale にならない）
コンテナの中で umount → mount した	成り立たない	Fargate のコンテナには mount の権限が無い。実機 S5：書き込み中は umount が「target is busy」。umount -l（遅延）で外して付け直しても、Java は外したマウント経由で書き続け、エラー 0・全行保存
EC2 起動タイプでホスト側の EFS マウントを外した	Stale にはならない	コンテナは古いマウントを持ち続ける。TLS の通信路（efs-proxy）が片付けられると「固まる」ことはありうるが、症状は Stale ではない

タスク定義の EFS 設定を変えて再デプロイした（外す→付ける）	間接的に成り立つ	再デプロイ中は新旧タスクが同時に動き、同じ app.log を開く。新タスクの起動時ロールオーバや、旧タスクの時刻ロールオーバが相手のファイルを消す → 1 位の原因
EFS 側のディレクトリ・アクセスポイント・根元を消して作り直した	成り立つ（3・4 位）	古いタスクの引換券が無効になる。アクセスポイントのルートは次のマウントで自動的に作り直されるので、新タスクは正常・古いタスクだけ Stale

■ 評価 「付け外し」は原因ではない。「デプロイの前後で起きた」は、新旧タスクの同時書き込み（1 位）か、デプロイに伴う削除・作り直し（3・4 位）を指している。まず 1 位を疑い、08_切り分けで確かめる。

■ 可能性の高い順（詳しくは 06_原因一覧）

順位	原因	可能性	この環境での当てはまり
1	複数タスクが同じログファイルに書き、Log4j2 のロールオーバ（改名→gz 圧縮→元ファイル削除・Delete）が他タスクの開いているファイルを消す	★★★★★	アプリログのリンク先はサービスの全タスクで共有。ローリング（100/200）・desiredCount≥2・ブルー/グリーンで必ず重なる
2	ログの片付け（日数・名前で消すバッチ、手動の find -delete）が、動いているタスクのファイル・ディレクトリを消す	★★★★☆	mid/ の定期削除を設計書で推奨。名前が起動時刻なので「古い名前＝止まったタスク」と誤解しやすい
3	デプロイ手順・起動スクリプト・手作業で、ログのディレクトリを消して作り直す・移す	★★★☆☆	「デプロイした後で」というタイミングと合う。古いタスクだけが Stale になる
4	マウントの根元（rootDirectory・アクセスポイントのルート・その上）を消して作り直す	★★☆☆☆	ls /mnt/logs 自体が Stale。そのタスクはもう直らず、入れ替えるしかない
5	同じタスクの front／back や別サービスが、別々のマウントで同じ場所を共有し、片方が消す	★★☆☆☆	別ボリューム・別アクセスポイントだと同じタスク内でも「別クライアント」扱いになる
6	EFS 自体の入れ替え（作り直し・別 FS への復元と切替・フェイルオーバ）、DataSync 等の削除同期	★☆☆☆☆	大きな作業の後だけ。CloudTrail に跡が残る
7	通信の断・再接続（マウントターゲット・SG・AZ 障害・efs-proxy）	★☆☆☆☆	主な症状は「固まる」「タイムアウト」「Input/output error」。Stale は削除と重なったときだけ
8	ご提示の仮説：動いているタスクで EFS のマウントを外して付け直した	☆☆☆☆☆	ECS では起きない操作。付け直してもエラーにならない（S5）
9	カーネル・efs-utils・JVM の不具合	☆☆☆☆☆	1〜8 を全部否定できたときだけ。版数を控えてサポートへ

■ 対処の早見表

区分	やること	かかる時間	シート

緊急（今日）	範囲を把握 → 証拠を残す → 消している側を止める → JBoss の reload かタスク入れ替え → 確認	30 分〜2 時間	09_緊急対応
恒久（数日）	タスクごとのログパス（1 ファイル 1 書き手）・止まったタスクだけ片付け・起動スクリプトから rm -rf を無くす・根元を守る・正本は CloudWatch Logs・Stale の監視	数日〜2 週間	10_恒久対策
運用（即日）	EFS 上の禁止操作の周知・片付けのルール・デプロイ中は片付けない・変更は「切り替えてから消す」	即日	11_運用回避

■ 以前の資料の訂正 「Log4j2_EFS_ローリングアップデート_FD競合_完全解説」5.5 では、圧縮後の削除で「B の FD が残っているので .nfs に化ける」と書いたが、消したのが別タスク（別の NFS クライアント）なら .nfs（silly rename）は働かない。EFS では B は次の書き込みで Stale file handle になる（同じ資料の 3.3「要検証」がこれ）。今回の例外はまさにこの形。.nfs になるのは同じタスク内で消した場合だけ（実機 S4）。

02　小学生にもわかるたとえ話

むずかしい言葉を使わずに、何が起きているかを学校のできごとにたとえる。最後に「たとえ」と「本当の名前」の対応表がある。

■ ① 引換券とロッカー（ファイルハンドル）

プールに行くと、荷物をロッカー 123 番に入れて「123 番の鍵（引換券）」をもらう。泳いでいるあいだ、あなたは「ロッカーの名前」ではなく「123 番の鍵」を持っている。

コンピューターも同じ。アプリが EFS のファイルを開くと、カーネル（コンピューターの係の人）は「app.log という名前」ではなく、「そのファイルそのものの番号札（ファイルハンドル）」を覚える。そのあとの「書いて」は全部この番号札で頼む。

もし、あなたが泳いでいるあいだに、ほかの人がロッカー 123 番の中身を捨てて、ロッカーごと片付けてしまったら？あなたが鍵を差しても「その鍵はもう古いよ（Stale）」と言われる。これが Stale file handle。

■ ② 家の机と、学校の共同倉庫（ローカルディスクと EFS のちがい）

家の机（ローカルディスク）	学校の共同倉庫（EFS）
お母さん（カーネル）は、あなたがノートを使っていることを知っている。	倉庫の係（EFS）は遠くにいて、たくさんの教室（タスク）から使われている。
だれかが「そのノート捨てといて」と言っても、あなたが使い終わるまで待ってから捨てる。	別の教室の子が「捨てて」と言うと、係はすぐに捨てる（あなたが使っているかは気にしない）。
だから、使っている途中で困ることはない（名前が見えなくなるだけ）。	あなたが次に「123 番のノートの続きに書いて」と頼むと「そのノートはもう無い」→ Stale。

■ ③ 自分で捨てたときは「忘れ物箱」（.nfsXXXX）

あなた自身（同じタスク）が、使っている途中のノートを「捨てて」と言ったときは、あなたの係（あなたのカーネル）が気をきかせて、ノートを「.nfs0000…」という名前の忘れ物箱に入れておき、あなたが使い終わった瞬間に捨てる。だからエラーにはならない。でも、使い終わったら中身ごと消えるので、書いた内容は残らない（実機 S4）。この気づかいができるのは「自分の係」だけ。別の教室（別のタスク）が捨てた場合は、だれも気づかってくれない。

■ ④ 道しるべ（シンボリックリンク）は悪くない

コンテナの中の /webapp/webapp9mf02/logs は「倉庫の front/inter-api/logs の棚はこちら →」という道しるべ。あなたは最初にノートを取りに行くとき 1 回だけ道しるべを見る。あとはノートを手に持っているので、道しるべは関係ない。だから道しるべは今回の犯人ではない。ただし、道しるべの向き先を「自分専用の棚」にすると、他の子とノートを取り合わなくなる（これが根本対策）。

■ ⑤ 棚の作り直しと、教室の建て直し

・棚（ログのディレクトリ）を作り直した場合：あとから来た子は名札を見て新しい棚にたどり着ける（Linux は名前を引き直してくれる）。でも、古い棚の前に立っていた子（開いているファイル・カレントディレクトリ）は、何もない所に立っていることになる（実機 S2）。

・教室（マウントの根元＝rootDirectory やアクセスポイントのルート）を建て直した場合：あなたの入館証は古い建物用なので、どの部屋にも入れない。新しい入館証をもらう（マウントし直す＝ECS ではタスクの入れ替え）まで何もできない（実機 S3）。

■ ⑥ 門を付け替えても、手に持ったノートは変わらない（ご提示の仮説）

学校の門（マウント）を付け替えても、あなたがすでに手に持っているノートはそのまま使える。門の付け替えは問題ではない。問題は「ノートを捨てること」。実機 S5 でも、書いている途中にマウントを外して付け直したが、エラーは 0 回だった。

■ ⑦ 2 人で 1 冊の日記帳（共有パスとロールオーバ）

クラスの A さんと B さんが「今日の日記」という 1 冊のノートに同時に書いている。夜中の 0 時を過ぎると、先に気づいた B さんが決まりどおり「今日の日記」を「昨日の日記」に名前を変え、小さく折りたたんで（gz 圧縮）箱にしまい、元のノートを捨て、新しい「今日の日記」を用意して書き始める。少し遅れて A さんも同じことをする。でも A さんが「今日の日記」だと思って片付けたのは、B さんが書き始めたばかりの新しいノートだった。B さんのペンは空中を書くことになる → Stale。しかも同じ名前の箱（.gz）に入れ直したので、前の日の日記まで上書きされてしまう。2 人が同じノートを使う限り、どちらかが毎回こうなる（実機 S6）。1 人 1 冊にすれば、何も起きない（実機 S7）。

■ たとえと本当の名前の対応

たとえ	本当の名前	説明
ロッカーの鍵・番号札	ファイルハンドル（NFS）／inode 番号	名前ではなく「そのもの」を表す番号。名前を変えても同じ
お母さん・係の人	カーネル（NFS クライアント）	アプリの代わりに EFS とやりとりする
学校の共同倉庫	Amazon EFS（NFS サーバ）	たくさんのタスクが同時に使う置き場
教室	ECS のタスク	タスクごとに別の係（別の NFS クライアント）がいる
忘れ物箱	.nfsXXXX（silly rename）	同じクライアントで使用中のファイルを消すと、一時的にこの名前になる
道しるべ	シンボリックリンク	開くときに 1 回だけたどる。原因ではない
入館証	マウント	根元が建て直されると使えなくなる
日記帳の名前を変えて箱にしまう	ロールオーバ（改名・圧縮・削除）	Log4j2 が時刻やサイズで行う
「その鍵は古いよ」	ESTALE（116）／NFS4ERR_STALE（70）	Java では IOException: Stale file handle

03　歴史と背景：なぜこのエラーがあり、なぜ今ここで起きるのか

Stale file handle は 1980 年代の NFS の設計から生まれた「わざと用意されたエラー」。それがコンテナ時代に「ログを共有ストレージに書く」設計と出会い、いま再び目立っている。

■ 年表

年	出来事	今回との関係
1970 年代	UNIX が「名前（ディレクトリの項目）」と「実体（inode）」を分ける。開いたファイルは実体に結びつき、名前を消しても閉じるまで実体は残る	ローカルディスクで「使用中のファイルを消しても平気」なのはこの設計のおかげ
1984〜1985 年	Sun Microsystems が NFS を開発し、SunOS 2.0 で出荷。サーバは「状態を持たない（stateless）」設計で、クライアントは「ファイルハンドル」で操作する	サーバが再起動しても同じ引換券で続きができる（実機 S9）。代わりにサーバは「誰が開いているか」を知らない
1985 年ごろ〜	「他のホストが消したファイルの引換券」を表すエラー ESTALE（Stale NFS file handle）が UNIX に加わる。クライアント側には silly rename（.nfsXXXX）の工夫	今回のエラーの直接の祖先。silly rename は「同じクライアント内」だけを守る
1989 年	RFC 1094（NFS v2）。NFSERR_STALE	ファイルハンドルは「ファイルシステム ID・inode 番号・世代番号」から作られる
1995 年	RFC 1813（NFS v3）。NFS3ERR_STALE（70）	検証では v3 も使用（状態を持たないので、消されるとすぐ Stale）
2000・2003 年	RFC 3010 → RFC 3530（NFSv4）。OPEN・CLOSE・ロックをサーバが覚える「状態を持つ」設計へ	ただし「開いているあいだは消さない」はサーバ実装しだい（Linux の knfsd は残す＝実機 S1 v4.1、EFS は Stale を返す）
2010 年	RFC 5661（NFSv4.1。2020 年に RFC 8881 へ改訂）。セッションなど	EFS の既定の版
2011 年	The Twelve-Factor App が「ログはイベントの流れ（標準出力）として扱い、ファイルの管理をアプリにさせない」と提唱	根本対策「正本は CloudWatch Logs」の考え方
2012〜2013 年	Linux 3.7〜3.8：パス名を使うシステムコールが ESTALE を受けたら、名前を引き直して 1 回やり直す仕組み（retry_estale）	ディレクトリの作り直しは「新しく開く」操作なら自動で回復（実機 S2）。開いている FD は対象外
2013 年	glibc（2.18）がメッセージを「Stale NFS file handle」→「Stale file handle」に変更（NFS 以外も使うため）	Java の例外メッセージはこの文字列。監視はこの語句で引っかける
2013〜2017 年	Docker 公開（2013）→ Amazon ECS（2014 発表・2015 GA）→ AWS Fargate（2017）	コンテナは使い捨て。止まるとエフェメラルストレージごとログが消える

2015〜2016 年	Amazon EFS 発表（2015-04）→ 一般提供（2016-06）。NFSv4.0／4.1 のマネージドサービス	多数のタスクから同じ場所を使える「共同倉庫」
2014〜2021 年	Log4j 2.0（2014-07）、Log4j 1 の保守終了（2015-08）、Log4Shell（2021-12）	Log4j2 は書けなくなっても自動で開き直さない（2021 年の要望 LOG4J2-3099 は Invalid で閉じられた）
2020-04	Fargate プラットフォーム 1.4.0 と ECS で EFS ボリュームが使えるように	コンテナのログ・データの置き場として EFS が一気に広まる
2020-11	AWS の技術者が Linux カーネルのメーリングリストで「EFS では、別のクライアントが消したファイルを閉じようとすると ESTALE が返る」と報告	EFS が「開いているファイルでも、他のクライアントが消せば Stale」を返す根拠
2021-03	ECS Exec（動いているコンテナに入って調べられる）	07_確認方法の土台
2023-09	OpenJDK 21（LTS）	現在の実行環境（Red Hat build of OpenJDK 21）
2024 年	JBoss EAP 8.0（2024-02）。efs-utils v2.0 で TLS の通信路が stunnel から efs-proxy（AWS 製）へ。1 クライアントあたり 1.5 GiB/s（2024-05）	efs-proxy 経由のマウントは 127.0.0.1:<ポート> に見え、マウントごとに別の NFS クライアントになる
2025 年	JBoss EAP 8.1。ECS の組み込みブルー/グリーン（2025-07）、リニア／カナリア（2025-10）	新旧タスクが「同時に動く時間」が長くなる方向。共有パスのままだと 1 位の原因が増える
2026 年	efs-utils 3.x、Log4j 2.26.1（2026-06）／2.25.5（2026-07）、UBI 9.8、OpenJDK 21.0.12（2026-08）	検証に使った版（14_検証結果）

■ なぜ「コンテナのログを EFS に書く」設計が生まれたのか（必要になった経緯）

1　コンテナは使い捨て

タスクが止まる・入れ替わると、コンテナの中（エフェメラルストレージ）に書いたログは消える。障害の調査や監査のためにログを残したい。

▼

2　readonlyRootFilesystem=true

セキュリティのためルートファイルシステムを読み取り専用にすると、そもそもコンテナの中に書けない。書ける場所（タスクのボリュームや EFS）へ逃がす必要がある。

▼

3　「全タスク共通の置き場」として EFS

EFS ならタスクが入れ替わってもログが残り、踏み台からも見られる。アプリ（WAR）は /webapp/…/logs に書く前提なので、シンボリックリンクで EFS へつなぐ。

▼

4　ローカルディスクの感覚のまま使う

「同じファイルにみんなで書く」「古いログは日数で消す」「ディレクトリを作り直して整理する」――ローカルでは問題にならなかった操作。

▼

5　NFS の約束ごとにぶつかる

NFS では、他のクライアントが開いているファイルを消すと、開いている側は Stale になる。さらに同時追記（O_APPEND）も NFS では安全でない。→ ログの消失と Stale file handle。

■ なぜ NFS は「他人が消すと Stale」なのか（設計の理由）

・初期の NFS は「サーバは状態を持たない」ことを最優先にした。サーバが落ちて戻っても、クライアントは同じ引換券で続きができる（壊れにくい）。

・その代わり、サーバは「どのクライアントがどのファイルを開いているか」を覚えない。だから「開いているあいだは消さない」というローカルの約束を守れない。

・引換券は名前ではなく「ファイルシステム ID・inode 番号・世代番号」なので、同じ名前で作り直しても別物になる（作り直しても直らない）。

・NFSv4 で「開いている」をサーバが覚えるようになったが、消されたファイルを残すかどうかは実装しだい。EFS は Stale を返す。

・クライアント側の救済策（silly rename）は「自分で消したとき」にしか効かない。複数のタスク（＝複数のクライアント）が同じ場所を使うコンテナ環境では、救済されない場面が増える。

■ 2024～2026 年の最新事情（今回に関係するもの）

・efs-utils は v2 で TLS の通信路を efs-proxy に置き換え（stunnel は互換用に残る）、v3 系（2026 年時点 3.3 系）でも改良が続く。TLS でマウントすると NFS の相手は 127.0.0.1:<ポート> になり、マウントごとに別クライアントとして扱われる。

・Fargate で EFS を使うと、タスクの中に EFS を管理する監督コンテナ（aws-fargate-supervisor）が作られる。マウントはタスクごと。

・ECS は 2025 年に組み込みのブルー/グリーン、リニア、カナリアを追加。いずれも新旧 2 組のタスクを並べて動かすので、共有パスの危険は増える。

・Log4j 2.25.0 で「ロールオーバに失敗したらファイルを開き直す」修正、2.25.5／2.26.1 で createOnDemand の修正。ただし「書き込みが Stale で失敗したら開き直す」機能は今も無い。

・ECS Managed Instances（2025 年～）でも EFS ボリュームを使える。仕組み（タスクごとのマウント・NFS の約束ごと）は同じ。

04　動作原理：引換券・削除・マウント・Log4j2・エラーの道筋

「名前」と「実体」と「引換券」の 3 つを分けて考えると、すべて説明できる。

■ 4-1　名前・実体・引換券

もの	何か	どこにある	名前を変える（rename）と	消す（unlink）と
名前（パス）	/webapp/webapp9mf02/logs/app.log のような文字列	ディレクトリの中の項目	変わる	なくなる
実体（inode）	ファイルの中身と属性。番号（inode 番号）で区別	EFS の中	変わらない（同じ実体）	ローカル：閉じるまで残る／NFS：消える
FD（ファイルディスクリプタ）	プロセスが「開いたファイル」を指す番号（例 45）	JVM の中	影響なし（実体を指している）	ローカル：書ける／NFS（別クライアントが消した）：Stale
NFS のファイルハンドル（引換券）	EFS が発行する「その実体」の証明（ファイルシステム ID・inode・世代番号から作る）	カーネルが覚えている	影響なし	無効になる → NFS4ERR_STALE

Log4j2 が app.log を開くと、カーネルは EFS から引換券をもらい、FD と結びつける。以後の書き込みは「名前」ではなく「引換券」で行う。だから名前を変えられても（rename）書き続けられる一方、実体を消されると引換券が無効になる。

■ 4-2 「消す」の扱い：ローカルディスクと NFS（EFS）のちがい

場面	消したあと、開いている側の書き込みは	理由	検証
ローカルディスク（ext4・xfs・コンテナの中）	成功する（見えないファイルに書かれ、閉じると消える）	カーネルが「開いている人」を数えていて、0 になるまで実体を残す	—
NFS：同じクライアント（同じタスク・同じマウント）で消した	成功する（.nfsXXXX に改名される）	自分のカーネルが「開いている」を知っているので silly rename する。閉じると中身ごと消える	S4
NFS：別クライアントが消した（EFS）	失敗する：Stale file handle	EFS は他のクライアントが開いていても実体を消し、引換券に STALE を返す	AWS 技術者の報告（2020）・ご質問の事象
NFS：別クライアントが消した（Linux knfsd・NFSv4.1）	成功する（ただし書いた行は消える）	knfsd は OPEN の状態を持つ間は実体を残す。名前は無いので、閉じると消える	S1 v4.1

NFS：別クライアントが消した（NFSv3）	失敗する：Stale file handle（約 3 秒後から）	v3 は状態を持たないので実体がすぐ消える	S1 v3

■ 大事な点　EFS では「誰かが開いているかどうか」に関係なく、別のクライアントが消せば終わり。ローカルディスクの感覚で「使用中でも消して大丈夫」と考えると事故になる。さらに Linux の knfsd では起きないので、「手元の NFS では大丈夫だった」は EFS での安全の証拠にならない。

■ 4-3　ECS で EFS がつながる仕組み（どこが NFS クライアントか）



back コンテナも同じボリュームを /mnt/logs に付けていれば、同じ NFS クライアントを共有する。別ボリューム・別アクセスポイントなら別クライアント。

・マウントするのはコンテナではなく、Fargate の監督コンテナ（aws-fargate-supervisor）や EC2 の ECS エージェント（ボリュームプラグイン）。コンテナにはバインドマウントで見せるだけなので、コンテナの中から外したり付け直したりはできない。

・タスクが違えば NFS クライアントも違う（Fargate はタスクごとに別の VM）。タスク A が開いているファイルをタスク B が消すと、A は救済されない（4-2 の「別クライアント」）。

・TLS（転送中の暗号化）を有効にすると、カーネルから見た相手は 127.0.0.1:<ポート>。マウントごとにポートが違うので、同じタスク（同じカーネル）でもボリュームが別なら別クライアント扱いになり、silly rename の保護は届かない。

・推奨のマウントオプションは nfsvers=4.1・hard・timeo=600・retrans=2・noresvport（efs-utils の既定）。hard なので、通信が切れると「エラー」ではなく「待つ（固まる）」。Stale とは症状がちがう。

・タスク定義の rootDirectory、またはアクセスポイントの Path が「根元」。アクセスポイントは、根元が無ければ次のマウント時に自動で作る（AWS 公式：If you delete the root directory, EFS recreates it the next time…）。そのため根元を消すと「新しいタスクは正常、古いタスクだけ全滅」になる。

■ 4-4　シンボリックリンクの役割（原因ではない）

・リンクは open（開く）のときに 1 回だけたどられる。/webapp/webapp9mf02/logs/app.log → /mnt/logs/front/inter-api/logs/app.log に置き換えて、そこで実体を開く。

・開いたあとの write（書く）は FD → 実体へ直接行く。リンクを消しても、張り替えても、開いているファイルには影響しない。

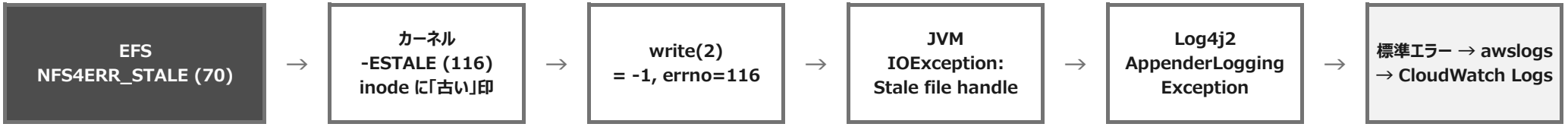
・ただしリンクは「どのディレクトリに書くか」を決める。リンク先をタスクごとのディレクトリにすれば、アプリを変えずに「1 ファイル 1 書き手」にできる（10_恒久対策）。

・readonlyRootFilesystem=true でもリンク自体はイメージに焼けるので、JBoss ログと同じ「二段リンク」（イメージのリンク → タスクローカルのリンク → EFS）が使える。

■ 4-5 Log4j2 がファイルに書く仕組み

動き	中身	Stale との関係
開く	FileManager が new FileOutputStream(ファイル, append=true)。O_APPEND（末尾に追記）で開く	引換券をもらう
書く	イベントごと（immediateFlush=true の既定）に write。FD は開きっぱなし	消されると次の write が Stale
ロールオーバ	時刻・サイズ・起動時（OnStartupTriggeringPolicy）に、閉じる → 改名 → 新しく作る。圧縮（.gz）なら別スレッドで圧縮後に元を削除。Delete で古いものを削除	共有パスだと、ここで他タスクのファイルを消す（1 位の原因）
失敗したとき	例外は StatusLogger へ（標準エラー → awslogs → CloudWatch Logs）。既定の ignoreExceptions=true ではアプリは止まらない	エラーは EFS のファイルではなく CloudWatch Logs に出る
開き直し	書き込み失敗では開き直さない（LOG4J2-3099 は Invalid）。次のロールオーバ・再設定・アプリ再起動で新しく開く。2.25.0 で「ロールオーバ失敗時は開き直す」修正	Stale は次のロールオーバまで続く（ログの欠落）
複数 JVM	同じファイルを複数の JVM で書く排他は無い（RollingFileAppender に locking は無い）	共有パスはそもそも想定外の使い方

■ 4-6 エラーが伝わる道筋



・Java のメッセージ「Stale file handle」は C ライブラリ（glibc）の strerror(116) の文字列そのもの。

・Log4j2 の出力例（実機 S6）：「main ERROR Unable to write to stream /webapp/webapp9mf02/logs/app.log for appender APP」→「org.apache.logging.log4j.core.appender.AppenderLoggingException: Error writing to stream …」→「Caused by: java.io.IOException: Stale file handle」。

・閉じるときに出る形：「main ERROR RollingFileManager … Unable to close stream: java.io.IOException: Stale file handle」（相手に消されたあと、自分のロールオーバで閉じようとした）。

・一度 Stale になった inode は、カーネルが「古い」と印を付けるので、以後の書き込みはサーバに聞かずに即失敗する。

■ 4-7 Linux が自動で直してくれる範囲

操作	自動で直るか	理由	検証
パス名で新しく開く・作る・調べる（open・stat・mkdir など）	直る（1 回やり直す）	ESTALE を受けると、名前を最初から引き直してやり直す（retry_estale、Linux 3.7〜）	S2
すでに開いている FD への書き込み・読み込み	直らない	名前を持っていないので引き直せない。アプリが開き直すしかない	S1・S6

カレントディレクトリ（cd した先）	直らない	同上（ls . が Stale）	S2
マウントの根元	直らない	根元はマウントの起点なので引き直す先が無い。マウントし直す（タスク入れ替え）しかない	S3

■ 4-8 エラーが少し遅れて出る理由（属性キャッシュと close-to-open）

・NFS クライアントは、ファイルの属性（サイズ・更新時刻）を数秒〜1 分キャッシュする（acregmin=3 秒・acregmax=60 秒、ディレクトリは 30〜60 秒）。

・そのあいだはサーバに確かめずに書き込み（ページキャッシュ）を受け付けるので、消されてもすぐにはエラーにならない。実機 S1 v3 では削除から約 3 秒後に最初の Stale。

・NFS は「開くとき（open）に必ずサーバへ確かめる」約束（close-to-open 一貫性）。診断スクリプトはこれを使い、読み取りで開き直してサーバの答えを取る（stat だけだとキャッシュにだまされる）。

・同じ理由で、他のタスクが書いた内容は、自分が開き直すまで見えないことがある（S5 で途中の行数が 0 に見えたのはこのため）。

■ 4-9 EFS ならではの性質（まとめ）

・NFSv4.0／4.1 で話す（既定 4.1）。ファイルハンドルは EFS が発行し（ファイルシステムごとに推測できない値）、ファイルが存在する限り同じ引換券で使える（NFS の設計どおり）。

・別クライアントが消したファイルへの操作は Stale になる（2020 年の AWS 技術者の報告）。開いているかどうかで守ってはくれない。
・「誰がどのファイルを消したか」の記録は残らない（CloudTrail に残るのはアクセスポイント・マウントターゲット等のマネジメント操作だけ）。消した側は周辺の記録から推定する。

・アクセスポイントは根元が無ければ次のマウントで作る。だから根元の作り直しは「新しいタスクだけ動く」形で表に出る。
・ライフサイクル管理（IA・アーカイブへの移動）や AWS Backup（復元は別ディレクトリ）は、ファイルハンドルを変えないので原因にならない。

05　動作イメージ：6 つの場面を 1 コマずつ

横に登場人物、縦に時間。濃い灰色のセルが失敗（エラー）、薄い灰色が要所。時刻は例（実機では 10 秒ごとのロールオーバに縮めて再現した）。

■ 場面 A（1 位の原因）：2 タスクが同じ app.log に書き、同じ時刻にロールオーバする

時刻のロールオーバは「境目を過ぎた最初のログ」で起きるので、2 タスクは数秒ずれて同じ境目でロールオーバする。先に回したタスクの新しいファイルを、あとから回したタスクが改名・圧縮・削除してしまう。

時刻	タスク A（JVM·Log4j2）	タスク B（JVM·Log4j2）	EFS 上の名前 → 実体	結果
00:00〜	app.log を開いて書く（実体 #1）	同じ app.log を開いて書く（同じ #1）	app.log → #1	1 つの実体に 2 タスクが追記（行が混ざる・欠ける）
01:00:00.2	（まだ #1 に書いている）	1 時を過ぎた最初のログ → ロールオーバ：#1 を閉じ、app.log を app-····-00.log に改名し、新しい app.log（#2）に書く	app-····-00.log → #1 ／ app.log → #2	
01:00:00.6	（まだ #1 に書いている）	別スレッドで app-····-00.log を gz に圧縮し、元（#1）を削除	app-····-00.log.gz ができる。#1 は削除	A がこの後 #1 に書いた行は、どこにも残らない
01:00:03	1 時を過ぎた最初のログ → ロールオーバ：#1 を閉じる → 失敗（Unable to close stream: Stale file handle）			A の書きかけの行が失われる
01:00:03	app.log（いまは B の #2）を app-····-00.log に改名し、新しい app.log（#3）に書く → A は回復	（#2 に書き続けている）	app-····-00.log → #2 ／ app.log → #3	
01:00:03.5	別スレッドで app-····-00.log（#2）を gz に圧縮して削除		app-····-00.log.gz を #2 の中身で上書き。#2 は削除	B が作った 0 時台の .gz が上書きされ、0 時台のログが丸ごと消えることがある
01:00:04〜		次の書き込みから毎回失敗：Unable to write to stream … Caused by: java.io.IOException: Stale file handle		B のログは次のロールオーバ（2 時）まで失われる
02:00		今度は B が先に回すと、あとから回した A が B の新しいファイルを消す		どちらかが毎回 Stale になる（実機 S6）

共有パスのままだと、どちらか一方だけを直しても止まらない。新旧タスクが重なるデプロイ中・desiredCount≥2 では毎回のロールオーバで起きうる。「先にロールオーバしたタスクが、次の境目まで Stale になる」「あとから回したタスクは Unable to close stream を出す」という出方になる。実機 S6 の結果：Stale の出現 A=12 B=10回 / 出力 A=225 B=225 / 残った A=61 B=103

■ 場面 B（2 位の原因）：片付けバッチが、動いているタスクのファイルを消す

時刻	タスク A の JVM（JBoss・Log4j2）	タスク A のカーネル	EFS	片付けバッチ（別ホスト・別タスク）
10/01 09:00	起動して error.log を開く（めったに書かない）	引換券 #7	error.log → #7	
10/01〜10/31	エラーが無いので書かない（更新時刻は古いまま）			
11/01 03:00				**find … -mtime +30 -delete（30日更新の無いファイルを消す）**
11/01 03:00			**#7 を削除（A が開いているとは知らない）**	
11/01 10:15	**障害が起き、error.log に書こうとする**	**#7 で WRITE／属性確認**	**#7 は無い → STALE**	
11/01 10:15	**IOException: Stale file handle（肝心のエラーログが残らない）**	**ESTALE**		

名前が起動時刻のディレクトリ（mid/20261001090000-xxxx）を「名前が古いから」と消す場合も同じ。長く動いているタスクほど名前は古い。

■ 場面 C（3 位の原因）：ログのディレクトリを消して作り直す

時刻	タスク A の JVM／シェル	タスク A のカーネル	EFS	作業者・別タスクの起動スクリプト
t0	logs/app.log を開いて書いている。シェルは cd logs 済み	logs=#20・app.log=#21 を覚える	logs(#20)/app.log(#21)	
t1			**logs(#20) と中身を削除 → 同じ名前で logs(#30)**	**rm -rf logs && mkdir logs**
t2	**開いている app.log（#21）に書く**	**#21 で WRITE**	**#21 は無い → STALE**	
t3	**新しく開く（ロールオーバ後の app.log など）**	**#20 で探す → STALE → 名前を最初から引き直して #30 を見つける**	**logs(#30)/app.log を作る**	
t3	**→ 新しく開く操作は自動で成功（S2）**			

t4	**シェルで ls .（カレントディレクトリ）→ Stale file handle**	**#20 で読む**	**STALE**	

■ 場面 D（4 位の原因）：マウントの根元（rootDirectory・アクセスポイントのルート）を作り直す

時刻	タスク A（古いタスク）	タスク A のカーネル	EFS	作業者・新しいタスク
t0	/mnt/logs を根元 /logs-root としてマウント済み	根元の引換券 #1	/logs-root(#1)	
t1			**/logs-root(#1) を削除 → 同じ名前で #9**	**踏み台で rm -rf … && mkdir …（またはアクセスポイントのルートを消す）**
t2	**ls /mnt/logs・mkdir・新しいログ・ロールオーバ**	**根元 #1 で探す → STALE。根元は引き直せない**	**#1 は無い**	
t2	**書き込みも新規作成も全部失敗（タスクが終わるまで続く）**			
t3			**根元 #9 で新しくマウント**	**新しいタスクが起動 → 正常（アクセスポイントなら根元を自動で作り直す）**
t4	**古いタスクを止めて入れ替える → 回復（S3）**			

■ 場面 E（参考）：同じタスクの中で消すと .nfs になり、エラーにならない

時刻	JBoss（JVM）	同じタスクのシェル・同じマウントの別コンテナ	カーネル（NFS クライアント）	EFS
t0	app.log（#21）を開いて書いている		#21 は自分のプロセスが開いていると知っている	app.log(#21)
t1		**rm app.log**	**削除の代わりに .nfs0000…0001 へ改名（silly rename）**	**app.log → .nfs0000…0001**
t2	書き続ける（エラーにならない）			.nfs…0001 に書かれる
t3	**閉じる（またはプロセス終了）**		**最後に閉じたので本当に削除**	**.nfs…0001 を削除（書いた中身も消える）**

■ 場面 F（ご提示の仮説）：書いている途中にマウントを外して付け直す

時刻	JBoss（JVM）	カーネル	EFS	操作
t0	app.log（#21）を開いて書いている	マウント M1 経由	#21	
t1		**使用中なので外せない：target is busy**		**umount /mnt/logs**
t2		**M1 を見えなくする（使用中の FD のために M1 は生きている）**		**umount -l /mnt/logs（遅延アンマウント）**
t3		**同じ場所に新しいマウント M2**		**mount …**
t4	書き続ける → M1 経由で #21 に届く（エラー 0）		#21 が伸び続ける（S5：全行保存）	
t5	新しく開くファイルは M2 経由			

■ ポイント　ECS では t1〜t3 の操作そのものが起きない（タスク定義を変えると新しいタスクになる）。付け直しは Stale の原因にならない。Stale は「実体が消された・作り直された」ときにだけ起きる。

06　原因一覧：可能性の高い順に全部

★ はこの環境（ECS ＋ EFS の共有パス ＋ Log4j2 ＋ ローリングデプロイ）での見込み。上から順に確かめれば、ほとんどは 1〜3 位で説明がつく。

■ 一覧

順位	原因	可能性	この環境での当てはまり
1	複数タスクが同じログファイルに書き、Log4j2 のロールオーバ（改名→gz 圧縮→元ファイル削除・Delete）が他タスクの開いているファイルを消す	★★★★★	アプリログのリンク先はサービスの全タスクで共有。ローリング（100/200）・desiredCount≥2・ブルー/グリーンで必ず重なる
2	ログの片付け（日数・名前で消すバッチ、手動の find -delete）が、動いているタスクのファイル・ディレクトリを消す	★★★★☆	mid/ の定期削除を設計書で推奨。名前が起動時刻なので「古い名前＝止まったタスク」と誤解しやすい
3	デプロイ手順・起動スクリプト・手作業で、ログのディレクトリを消して作り直す・移す	★★★☆☆	「デプロイした後で」というタイミングと合う。古いタスクだけが Stale になる
4	マウントの根元（rootDirectory・アクセスポイントのルート・その上）を消して作り直す	★★☆☆☆	ls /mnt/logs 自体が Stale。そのタスクはもう直らず、入れ替えるしかない
5	同じタスクの front／back や別サービスが、別々のマウントで同じ場所を共有し、片方が消す	★★☆☆☆	別ボリューム・別アクセスポイントだと同じタスク内でも「別クライアント」扱いになる
6	EFS 自体の入れ替え（作り直し・別 FS への復元と切替・フェイルオーバ）、DataSync 等の削除同期	★☆☆☆☆	大きな作業の後だけ。CloudTrail に跡が残る
7	通信の断・再接続（マウントターゲット・SG・AZ 障害・efs-proxy）	★☆☆☆☆	主な症状は「固まる」「タイムアウト」「Input/output error」。Stale は削除と重なったときだけ
8	ご提示の仮説：動いているタスクで EFS のマウントを外して付け直した	☆☆☆☆☆	ECS では起きない操作。付け直してもエラーにならない（S5）
9	カーネル・efs-utils・JVM の不具合	☆☆☆☆☆	1〜8 を全部否定できたときだけ。版数を控えてサポートへ

■ 原因 × 対処の早見表（緊急で直す／根本対応／運用で回避）

順位	原因（短く）	緊急で直す	根本対応	運用で回避

1	共有パス＋ロールオーバ	消した側を止め、影響タスクを 1 つずつ JBoss reload／入れ替え（ロールオーバ時刻を避ける）	アプリログをタスクごとのディレクトリへ（二段リンク）。Delete は自分のディレクトリだけ。正本は CloudWatch Logs	デプロイをロールオーバ時刻から離す。起動時ロールオーバを外す
2	片付けが使用中を消す	バッチを止め、影響タスクを reload／入れ替え	efs-log-housekeeping.sh（ECS に問い合わせ、止まったタスクだけ消す。.owner で持ち主判定）	保存期間＋停止済みだけ消す。デプロイ中は片付けない。まず -N で判定
3	手順・起動スクリプトの作り直し	その手順を止め、影響タスクを reload／入れ替え	EFS 上は mkdir -p だけ。rm -rf・mv・作り直しを手順から無くす	構成変更は「新しいパスへ切り替えてから古い方を最後に消す」
4	根元の作り直し	タスクを入れ替える（reload では直らない）	根元はアクセスポイントで作り、消せる人を IAM・FS ポリシーで絞る	根元・アクセスポイントを触るのはサービスを 0 にしてから
5	front／back・サービス間の共有	影響タスクを reload／入れ替え	コンテナ・サービスごとにディレクトリを分ける	共有するなら同じボリュームを使う
6	EFS の入れ替え・削除同期	タスクを入れ替える	大きな作業は新タスクで切り替える手順に	ログ置き場を同期の削除対象にしない
7	通信の断	SG・マウントターゲットを直し、固まったタスクを入れ替え	noresvport（既定）・変更は計画作業に	変更前後で estale-diag の「応答なし」を確認
8	マウントの付け直し（仮説）	対処不要（原因ではない）	―	―
9	不具合	証拠をそろえてサポートへ	版数を最新に	―

■ 原因ごとの詳細（仕組み・見分け方・緊急／恒久／運用）

▸ 1 位 ★★★★★ 複数タスクが同じログファイルに書き、Log4j2 のロールオーバが他タスクのファイルを消す

項目	内容
起き方	同じパス（/mnt/logs/front/inter-api/logs/app.log）を複数のタスクが開く。どれかのタスクのロールオーバが①改名（rename：移動先に同名があれば上書き＝相手の実体を消す）②圧縮後の元ファイル削除（.gz・.zip）③Delete（古いものを消す）④DefaultRolloverStrategy の max＋%i（古い番号を消す）のどれかで、別タスクが開いている実体を消す。
この環境での当てはまり	アプリログのリンク先はサービスの全タスクで共有（JBoss のログはタスクごとに分けてあるが、アプリログは共有のまま）。ローリングデプロイ（最小 100%・最大 200%）、desiredCount≥2、スケールアウト、ブルー/グリーン・カナリア・リニアで必ず複数タスクが重なる。
「起動すると出る」場合	新しいタスクの起動時にロールオーバが走る設定（OnStartupTriggeringPolicy、時刻ポリシーで前の期間のファイルを起動直後に回す、JBoss の rotate-on-boot）だと、デプロイのたびに旧タスクのファイルを消す。逆に旧タスクの時刻ロールオーバが新タスクのファイルを消すこともある。

症状の特徴	毎時 0 分・毎日 0 時（またはサイズの閾値）の直後から出て、自分の次のロールオーバで止まる。先にロールオーバしたタスクが「Unable to write to stream … Stale file handle」を次の境目まで出し続け、あとから回したタスクは「Unable to close stream … Stale」を 1 回出す。複数タスクで交互に出る。.gz の上書き（前の期間のログが丸ごと消える）・行の欠落・行の混ざり（O_APPEND は NFS では原子的でない）。
確かめ方	07-1 の Logs Insights で Stale の時刻とタスクを並べる → ロールオーバ時刻と一致するか。その時刻に同じパスを開いていたタスクが 2 つ以上あったか（07-5）。log4j2.xml の fileName が共有パスか、filePattern が .gz、Delete・max の有無（07-6）。estale-diag.sh で STALE／REPLACED。
緊急対応	影響タスクを JBoss の reload かタスク入れ替えで開き直す。入れ替えはロールオーバ時刻を避け、1 タスクずつ（重なりで再発するため）。
恒久対策	1 ファイル 1 書き手：アプリログのリンク先をタスクごとのディレクトリへ（10_恒久対策 P1）。Delete は自分のディレクトリだけ。正本は CloudWatch Logs。
運用回避	（恒久対策までのつなぎ）デプロイをロールオーバ時刻（0 時・毎時 0 分）から離す。起動時ロールオーバの設定を外す。desiredCount を一時的に 1 にできるなら重なりが減る。
検証	S6（v3）：Stale の出現 A=12 B=10回 / 出力 A=225 B=225 / 残った A=61 B=103。S6（v4.1・knfsd）：Stale の出現 A=0 B=0回 / 出力 A=225 B=225 / 残った A=49 B=175。S7（タスクごと）：Stale A=0 B=0 / 出力 A=225 B=225 / 残った A=225 B=225

▸ **2 位 ★★★★☆ ログの片付けが、動いているタスクのファイル・ディレクトリを消す**

項目	内容
起き方	cron・EventBridge Scheduler・Lambda・別の ECS タスク・踏み台の手作業が「更新から N 日」「名前（起動時刻）が古い」で消す。めったに書かないログ（error.log）や、長く動いているタスクのディレクトリ（mid/<起動時刻-乱数>）は「古い」と判定される。
この環境での当てはまり	設計書（ECS_EFS_Dockerfile_Symboliclink_lite）で mid/ 配下の定期削除バッチを推奨し、名前が起動時刻なので世代削除しやすいとしている。名前の古さは「止まった」ことを意味しない。
症状の特徴	バッチの実行時刻の直後から出る。長寿命のタスク・めったに書かないログで出る。そのタスクは次に開き直すまで直らない。
確かめ方	バッチの実行履歴（スケジュールの時刻・ログ）と Stale の最初の時刻を比べる。消されたファイル・ディレクトリの持ち主タスクが、その時点で RUNNING だったか（describe-tasks）。
緊急対応	バッチを止める（スケジュールの無効化）。影響タスクを reload／入れ替え。
恒久対策	片付けは ECS に生きているタスクを問い合わせ、止まったタスクのものだけ消す（tools/efs-log-housekeeping.sh：RUNNING・停止処理中・24 時間以内の更新・.nfs あり・リンク先は必ず残す。RUNNING 0 件なら中止）。持ち主はディレクトリの .owner（TaskARN）で判定。
運用回避	デプロイ中は片付けない。消すのは「保存期間を過ぎ、かつ止まったタスク」のものだけ。共有ディレクトリ直下のファイルを日数で消さない。

▸ **3 位 ★★★☆☆ デプロイ手順・起動スクリプト・手作業でディレクトリを消して作り直す・移す**

項目	内容

起き方	「ログを空にしてから起動」「整理のため logs を logs.bak に移して作り直す」「Ansible 等で一度消してから作る」――名前が同じでも実体（inode）が変わるので、古いタスクが開いているものはすべて Stale。
この環境での当てはまり	「アプリをデプロイした後で…」という時系列と合う。いまの entrypoint は mkdir -p だけで消さないが、手順書・別の仕組み・手作業が無いかを確かめる。
症状の特徴	作業の直後から、古いタスクだけが Stale。新しいタスクは正常。新しく開く操作は自動で回復するが、開いている FD・カレントディレクトリは直らない（S2）。
確かめ方	ディレクトリの inode が変わっていないか（stat -c %i を前後・タスク間で比べる）。作業ログ・踏み台のシェル履歴・デプロイの手順。
緊急対応	影響タスクの reload／入れ替え。
恒久対策	起動スクリプト・手順から EFS 上の rm -rf・mv・作り直しを無くす（mkdir -p だけ）。レビューの観点に入れる。
運用回避	構成変更は「新しいパスへ切り替えてから、古い方を最後に消す」（11_運用回避）。
検証	S2：cwd=ESTALE / パス名=OK / 新規作成=OK

▸ **4 位 ★★☆☆☆ マウントの根元（rootDirectory・アクセスポイントのルート・その上）を作り直す**

項目	内容
起き方	根元（例 /logs-root、アクセスポイントの Path）やその上のディレクトリを消して同じ名前で作り直す。アクセスポイントを消して作り直す。マウントしているタスクの根元の引換券が無効になり、そのマウントの下は全部 Stale。根元は引き直せないので直らない。
症状の特徴	ls /mnt/logs・stat /mnt/logs が Stale。新しいファイルも作れない（リンク経由だと No such file or directory になることもある：S3）。そのタスクの EFS へのログが全部止まる。新しく起動したタスクは正常（アクセスポイントは根元を自動で作り直す）。
確かめ方	コンテナの中で stat /mnt/logs。CloudTrail（DeleteAccessPoint・CreateAccessPoint）。踏み台でのディレクトリ操作。
緊急対応	タスクの入れ替え（stop-task・force-new-deployment）。reload では直らない。
恒久対策	根元はアクセスポイント（作成時に所有者・権限を指定）で作り、運用で消さない。踏み台からルートを書き込みでマウントできる人を IAM・ファイルシステムポリシーで絞る。
運用回避	根元・アクセスポイントを触るのは、そのボリュームを使うサービスを 0 にしてから。
検証	S3（v4.1）：マウント中は ESTALE / 再マウントで回復 / Java: SUMMARY writes=67 errors=0 firstError=-／S3（v3）：マウント中は ESTALE / 再マウントで回復 / Java: SUMMARY writes=67 errors=57 firstError=14:32:39.388 seq=11

▸ **5 位 ★★☆☆☆ 同じタスクの front／back や別サービスが、別々のマウントで同じ場所を共有し、片方が消す**

項目	内容
起き方	front と back が別のボリューム定義（別アクセスポイント・別 rootDirectory）で同じディレクトリを見ていると、同じタスクでも別の NFS クライアント（TLS ならポート違い）になり、片方が消したファイルはもう片方で Stale。back のリンクが誤って front のディレクトリを指す、別サービスが同じ場所を使う、も同じ。

確かめ方	タスク定義の volumes と mountPoints（ボリューム名・rootDirectory・accessPointId）。各コンテナで readlink -f /webapp/*/logs。
対処	コンテナ・サービスごとにディレクトリを分ける（1 ファイル 1 書き手）。同じ場所を共有するなら同じボリュームを使う。

▸ **6 位 ★☆☆☆☆ EFS 自体の入れ替え・削除同期**

項目	内容
起き方	ファイルシステムの作り直し、バックアップから別の FS へ復元して切り替え、レプリケーションのフェイルオーバ、DataSync や rsync --delete でログ置き場を同期先にする。
確かめ方	CloudTrail（CreateFileSystem・DeleteFileSystem・CreateReplicationConfiguration・StartTaskExecution など）、作業記録。
対処	大きな作業はサービスを止める・新しいタスクで切り替える手順にする。ログ置き場を同期の削除対象にしない。

▸ **7 位 ★☆☆☆☆ 通信の断・再接続**

項目	内容
起き方	マウントターゲットの削除・作り直し、セキュリティグループ／NACL の変更、AZ 障害、efs-proxy の停止。
症状の特徴	Stale ではなく「固まる（hard マウントで待ち続ける）」「タイムアウト」「Input/output error」。Stale は、切れている間に削除が重なったときだけ。
確かめ方	estale-diag.sh の「応答なし」、/proc/self/mountstats の tcp 接続回数（2 以上なら再接続あり）、CloudTrail（DeleteMountTarget 等）、VPC フローログ。
対処	noresvport（efs-utils の既定）を使う。マウントターゲット・SG の変更は計画作業に。

▸ **8 位 ☆☆☆☆☆ ご提示の仮説：動いているタスクで EFS のマウントを外して付け直した**

項目	内容
評価	ECS は動いているタスクのボリュームを付け外ししない。Fargate ではコンテナから mount／umount できない。仮に付け直しても、開いている FD は外したマウント経由で書き続けるので Stale にならない（S5：SUMMARY writes=67 errors=0 firstError=- / サーバ上の行数=67）。
読み替えると成り立つもの	「マウントの付け外し」＝「タスク定義を変えて再デプロイ」なら 1 位の引き金、「EFS 側の作り直し」なら 3・4 位。

▸ **9 位 ☆☆☆☆☆ カーネル・efs-utils・JVM の不具合**

項目	内容
評価	1〜8 をすべて否定できたときだけ疑う。Fargate のカーネル・efs-utils は AWS 管理なので利用者は変えられない。
やること	プラットフォーム版（Fargate 1.4.0 / LATEST）、EC2 なら AMI と amazon-efs-utils の版、Log4j2 の版、時刻・タスク ID・estale-diag.sh の出力を控えて AWS サポートへ。

■ 原因ではないもの（疑わなくてよい）

よく疑われるもの	原因でない理由
シンボリックリンク	開くときに 1 回たどるだけ。開いたあとは実体に直接書く（4-4）
UBI 9.8・OpenJDK 21・JBoss EAP 8.1 の版	NFS のやりとりはホスト（Fargate の VM・EC2）のカーネル。コンテナの OS は関係しない
EFS のライフサイクル（IA・アーカイブ）	保存クラスの移動はファイルハンドルを変えない
AWS Backup	同じ FS への復元は別ディレクトリ（aws-backup-restore_…）に作る。既存ファイルは変えない
スループットモード・性能	遅くなることはあっても Stale にはならない
Log4j2 の非同期化（AsyncAppender・AsyncLogger）	JVM の中の順番の話。他タスクとの競合には効かないし、原因にもならない
ECS Exec でコンテナに入ったこと	見るだけなら影響しない（中で rm や mv をしなければ）
タスクの CPU・メモリ	関係しない

■ 似ているが別のエラー（取り違えないために）

エラー文	意味	Stale との違い
No such file or directory（ENOENT／FileNotFoundException）	その名前が無い	親ディレクトリが消えた・リンク先が未作成。根元が Stale の下でも出ることがある（S3）
Input/output error（EIO）	通信や状態回復の失敗、soft マウントのタイムアウト	通信の問題（7 位）を疑う
応答なし・固まる・Connection timed out	hard マウントがサーバの返事を待っている	Stale ではない。SG・マウントターゲット・efs-proxy
Permission denied（EACCES）	権限が無い	アクセスポイントの uid/gid、IAM の ClientWrite、ディレクトリの権限
Read-only file system（EROFS）	読み取り専用の場所に書いた	readonlyRootFilesystem の領域、読み取り専用マウント
Device or resource busy（EBUSY）	使用中	.nfs ファイルがあるディレクトリを消そうとした（同じクライアントで使用中）
Disk quota exceeded（EDQUOT）	EFS の上限	1 インスタンスで同時に開けるファイル数・ロック数の上限など
Too many open files（EMFILE）	プロセスの FD 上限	ulimit。開き直しを繰り返して FD を漏らしていないか

07　確認方法：事象を確かめ、証拠を集める

エラーは EFS のファイルではなく CloudWatch Logs（コンテナの標準エラー）に出る。「いつ・どのタスク・どのファイル」をそろえ、コンテナの中で「根元か、ファイルか」を見分け、最後に「消した側」を探す。

■ 07-1　CloudWatch Logs で「いつ・どのタスク・どのコンテナ」

awslogs のログストリーム名は「プレフィックス/コンテナ名/タスク ID」。Logs Insights で、対象サービスのロググループ（例 /ecs/interapi）を選んで実行する。

Logs Insights：Stale を出したストリーム（タスク×コンテナ）ごとの件数と時刻

```
fields @timestamp, @logStream, @message
| filter @message like /Stale file handle/
| stats count(*) as n, min(@timestamp) as first, max(@timestamp) as last by @logStream
| sort first asc
```

Logs Insights：前後の流れ（書けない・閉じられない・作れない）を時刻順に

```
fields @timestamp, @logStream, @message
| filter @message like /Stale file handle|Unable to write to stream|Unable to close stream|AppenderLoggingException|FileNotFoundException/
| sort @timestamp asc
| limit 500
```

・最初の時刻が「毎時 0 分・0 時の直後」→ 1 位（ロールオーバ）。「バッチの時刻」→ 2 位。「作業・デプロイの直後」→ 3・4 位。

・複数のタスクで交互に出ている → 1 位（共有パス）。

・1 つのタスクだけ、しかもそのタスクのログが全部止まっている → 4 位（根元）を疑い、07-3 で stat /mnt/logs。

■ 07-2　スタックトレースの形で「どの操作で失敗したか」

見える形	失敗した操作	意味
at java.io.FileOutputStream.writeBytes（Native Method）／Unable to write to stream … Caused by: IOException: Stale file handle	開いているファイルへの書き込み	そのファイル（実体）が他のクライアントに消された
RollingFileManager … Unable to close stream: java.io.IOException: Stale file handle	ロールオーバで閉じるとき	閉じる前に相手に消されていた（共有パスの典型）
FileNotFoundException: …/app.log (Stale file handle) ／ FileOutputStream.open0	新しく開くとき	ディレクトリや根元が Stale（3・4 位）

java.nio.file.FileSystemException: …: Stale file handle（Files.move など）	ロールオーバの改名・削除	改名先・元が他から消された
FileNotFoundException: …/app.log (No such file or directory)	新しく開くとき	親ディレクトリが無い（消された・未作成）。根元の Stale でも出うる

■ 07-3 コンテナの中で確かめる（ECS Exec）

ECS Exec でフロント／バックのコンテナに入る。同梱の tools/estale-diag.sh を使うのが確実（下の 3 通りのどれかで実行）。すぐに見たいときは、その下の「貼り付け用」を対話シェルに貼る。どちらもファイルを変更しない。

コンテナに入る（対話）

```
aws ecs execute-command --cluster <クラスター> --task <タスクID> --container <コンテナ名> --interactive --command "/bin/bash"
```

estale-diag.sh の実行方法（3 通り）

```
# 1) イメージに入れておく（次のリリースから）：COPY tools/estale-diag.sh /usr/local/bin/estale-diag.sh
bash /usr/local/bin/estale-diag.sh            # 既定: NFS マウントを自動検出し、Java の FD を全部調べる
# 2) EFS 上に置く（踏み台や健全なタスクから /mnt/logs/_tools/ に置く）。根元が Stale のタスクでは読めない＝それ自体が判定
bash /mnt/logs/_tools/estale-diag.sh --probe  # --probe は一時ファイルを作って消す書き込みテストも行う
# 3) 下の「貼り付け用」を対話シェルに貼る
```

貼り付け用：根元・Java の FD・.nfs・再接続をまとめて確かめる

```
M=/mnt/logs
stat -c '根元 OK inode=%i' "$M" || echo '→ 根元が Stale: このタスクは入れ替えるしかない'
for P in /proc/[0-9]*; do case "$(readlink $P/exe 2>/dev/null)" in */java) J=${P#/proc/};; esac; done; echo "java pid=$J"

for F in /proc/$J/fd/*; do T=$(readlink $F); case "$T" in $M/*) if (: < $F) 2>/dev/null; then echo "OK  $T"; else echo "NG  $T  $( (: < $F) 2>&1 | sed 's/.*: //')"; fi;; esac; done

ls -la /webapp/*/logs/ | head -20
find -L /webapp/*/logs -maxdepth 3 -name '.nfs*' 2>/dev/null | head
grep -A9 "mounted on $M " /proc/self/mountstats | grep -E 'age:|xprt:'
```

結果	意味	次の一手
stat /mnt/logs が Stale file handle	根元が作り直された（4・6 位）	このタスクは入れ替える。他のタスクも同じか確かめる
FD が NG（Stale file handle）	開いているファイルが他のクライアントに消された（1〜3 位）	消した側を探す（07-5〜07-7）。JBoss reload／入れ替え
FD が NG（No such file or directory）	名前がサーバに無い（消された・改名された）	同上。EFS では次の書き込みで Stale になる
estale-diag で REPLACED	名前は別の実体を指している（他タスクがロールオーバした跡）	ログが想定と別のファイルに入っている。共有パスの証拠
.nfsXXXX がある	同じタスク内で使用中のファイルが消された	閉じると中身ごと消える。消した処理を探す

tcp接続回数が 2 以上	通信が一度切れて再接続した	時刻が Stale と重なるなら 7 位も視野に

・stat だけだと NFS のキャッシュの古い答えが返ることがある。開き直し（: < /proc/<pid>/fd/<n>）はサーバに確かめに行くので確実（4-8）。

・NFSv4.0 でマウントしている場合、改名されただけのファイルを NG と表示することがある（名前で開き直すため）。EFS の既定（4.1）では起きない。NFSv3 では、他のクライアントの改名に気づくまで最大 30 秒ほど（ディレクトリの属性キャッシュ）かかる。

・root で実行する（JBoss が別ユーザーでも FD を読めるように）。ECS Exec のコマンドは、コンテナのユーザー指定にかかわらず root で動く（AWS 公式）。

■ 注意：readonlyRootFilesystem=true と ECS Exec　AWS 公式は「SSM エージェントがコンテナのファイルシステムに書く必要があるため、readonlyRootFilesystem で読み取り専用にしたコンテナでの ECS Exec は非対応」としている。入れないときは ① CloudWatch Logs（07-1・07-2）と ECS の記録（07-5）だけで切り分ける ② 踏み台から EFS 側を見る（ファイル・ディレクトリの有無、stat -c %i の inode、.nfs の有無） ③ 調査用に、同じイメージで readonlyRootFilesystem を外したタスク定義のタスクを 1 つ起動し、そこで 07-3 を行う（同じ EFS・同じパスなら根元の状態は同じに見える） ④ 次のリリースから estale-diag.sh をイメージに入れ、ヘルスチェックや定期実行の出力を CloudWatch Logs に出す。

■ 07-4　ほかのタスクでも同じか（全タスク・全コンテナ）

サービスの動いているタスクを並べ、1 つずつ確かめる

```
aws ecs list-tasks --cluster <クラスター> --service-name <サービス> --desired-status RUNNING --query 'taskArns' --output text
# タスクごとに front／back で 07-3 を実行（または estale-diag.sh -q で判定だけ）
```

■ 07-5　そのとき同時に動いていたタスク（重なり）

デプロイの流れ・重なっていたタスク

```
aws ecs describe-services --cluster <クラスター> --services <サービス> --query 'services[0].events[:30].[createdAt,message]' --output text
aws ecs describe-services --cluster <クラスター> --services <サービス> --query 'services[0].[desiredCount,deploymentConfiguration]'

aws ecs list-tasks --cluster <クラスター> --service-name <サービス> --desired-status STOPPED --query 'taskArns' --output text   # 止まって約 1 時間以内

aws ecs describe-tasks --cluster <クラスター> --tasks <ID1> <ID2> --query 'tasks[].[taskArn,lastStatus,startedAt,stoppingAt,stoppedAt,stoppedReason]' --output table
```

Stale の時刻に、同じパスへ書くタスクが 2 つ以上 RUNNING だったなら 1 位がほぼ確定。重なりの長さは「新タスクの起動時間＋ALB の登録解除の待ち（既定 300 秒）＋stopTimeout」。

■ 07-6　Log4j2・JBoss の設定（ロールオーバで消す設定があるか）

コンテナの中で設定ファイルを見る

```
find / -name 'log4j2*.xml' -path '*WEB-INF*' 2>/dev/null
grep -nE 'fileName|filePattern|OnStartup|TimeBased|SizeBased|Delete|max=|createOnDemand|ignoreExceptions' <見つかった log4j2.xml>
/opt/jboss-eap/bin/jboss-cli.sh --connect --command='/subsystem=logging:read-resource(recursive=true)' | grep -E 'file|rotate|suffix|max-backup'
```

設定	共有パスでの危険
fileName が共有パス（/webapp/…/logs/app.log）	全タスクが同じ実体を開く（1 位の前提）
filePattern が .gz／.zip	圧縮後に元ファイルを削除 → 相手が Stale
Delete アクション（basePath が共有）	相手のファイルを消す
DefaultRolloverStrategy の max と %i	古い番号のファイルを消す・改名の上書き
OnStartupTriggeringPolicy／JBoss の rotate-on-boot	起動のたびに相手のファイルを回す（デプロイで起きる）

■ 07-7　消した側を探す（EFS は「誰が消したか」を記録しない）

・他タスクのロールオーバ：Stale の時刻とロールオーバの時刻、重なっていたタスク（07-5）。

・片付けバッチ：EventBridge Scheduler・cron・Lambda・ECS スケジュールタスクの実行時刻とログ。

・起動スクリプト・手順：rm・mv・ディレクトリの作り直しが無いか。タスクの起動時刻との一致。

・手作業：踏み台のシェル履歴（~/.bash_history）、作業記録、Session Manager のログ。

・EFS・ECS の設定変更：CloudTrail（下のコマンド）。

・EFS のメトリクス：ClientConnections（つながっているクライアント数。踏み台・バッチが来た時刻に増える）、MetadataIOBytes（大量の削除・改名で山ができる）。

CloudTrail と EFS のメトリクス

```
aws cloudtrail lookup-events --start-time <開始UTC> --end-time <終了UTC> --lookup-attributes AttributeKey=EventSource,AttributeValue=elasticfilesystem.amazonaws.com --query 'Events[].[EventTime,EventName,Username]' --output table
aws cloudtrail lookup-events --start-time <開始UTC> --end-time <終了UTC> --lookup-attributes AttributeKey=EventSource,AttributeValue=ecs.amazonaws.com --query 'Events[?EventName==`UpdateService`||EventName==`RegisterTaskDefinition`].[EventTime,EventName,Username]' --output table
aws cloudwatch get-metric-statistics --namespace AWS/EFS --metric-name ClientConnections --dimensions Name=FileSystemId,Value=<fs-ID> --start-time <開始> --end-time <終了> --period 300 --statistics Sum
aws cloudwatch get-metric-statistics --namespace AWS/EFS --metric-name MetadataIOBytes --dimensions Name=FileSystemId,Value=<fs-ID> --start-time <開始> --end-time <終了> --period 300 --statistics Sum
```

■ 07-8　EFS で実際に起きるかを確かめる（検証環境で）

本番のログではなく、検証用のディレクトリで行う。2 つのタスク（または 2 台の EC2）から同じ EFS を使う。

2 つのクライアントで「開いているファイルを消す」

```
# タスク A（書き続ける。FD 3 を開いたまま）
mkdir -p /mnt/logs/_estale_test && exec 3>>/mnt/logs/_estale_test/t.log
while sleep 1; do date >&3 || { echo "書けなくなった（rc=$?）"; break; }; done
# タスク B（A が開いているファイルを消す）
rm /mnt/logs/_estale_test/t.log
# → 数秒後に A で「date: write error: Stale file handle」が出れば、EFS でも同じことが起きる
```

```
# 後片付け（A で）：exec 3>&-; rm -rf /mnt/logs/_estale_test
```

08　切り分け：はい／いいえで原因を絞る

上から順に答える。各段の「はい」で原因と対処が決まる。迷ったら 07 のコマンドで事実を集めてから戻る。

■ 問題の切り分け手順

段	問い（どう確かめるか）	はい のとき	いいえ のとき
1	エラーは「Stale file handle」か？（「固まる」「タイムアウト」「Input/output error」ではないか）	2 へ	通信の問題（7 位）を先に調べる：estale-diag の「応答なし」、SG・マウントターゲット・efs-proxy
2	コンテナの中で stat /mnt/logs が Stale か？（07-3）	根元の作り直し（4 位）か EFS の入れ替え（6 位）→ タスク入れ替え必須。CloudTrail を見る	3 へ
3	ls /webapp/…/logs/（リンク先）が Stale・No such file か？	ログのディレクトリが消された・作り直された（3 位・2 位）。新しく開く操作は自動で戻るが、開いている FD は直らない	4 へ
4	Java の FD を開き直すと NG（Stale／No such file）か？	開いているファイルが他のクライアントに消された → 5 へ（誰が消したか）	いまは正常。過去の事象なら 07-1 の時刻から 5 へ
5	その時刻に、同じパスへ書くタスクが 2 つ以上動いていたか？（デプロイ中・desiredCount≥2）	1 位（共有パス＋ロールオーバ）がほぼ確定。時刻がロールオーバ時刻・起動直後と合うか確認	6 へ
6	時刻が片付けバッチ・手作業・デプロイ手順と一致するか？	2 位（片付け）・3 位（手順・手作業）	7 へ
7	同じタスクの別コンテナ・別サービスが、別のマウントで同じ場所を使っていないか？	5 位	8 へ
8	CloudTrail に EFS（アクセスポイント・マウントターゲット・FS）の変更があるか？	4・6・7 位	9 位（不具合）を疑い、版数と証拠をそろえてサポートへ

■ 時刻のパターンで見分ける

Stale が出始めた時刻・出方	疑う原因
毎時 0 分・毎日 0 時（またはサイズの閾値）の直後から出て、次のロールオーバで止まる。タスク間で交互	1 位（共有パス＋ロールオーバ）
デプロイ・スケールアウトで新タスクが起動した直後から、古いタスクで出る	1 位（起動時ロールオーバ）・3 位（起動スクリプト）

毎日決まった時刻（片付けの時刻）から。長く動いているタスク・めったに書かないログ	2 位（片付け）
作業の直後から。古いタスクだけ。新しいタスクは正常	3 位・4 位
そのタスクのログが全部止まる。ls /mnt/logs が Stale	4 位（根元）
不規則・ときどき	1 位（サイズによるロールオーバ）・2 位
Stale ではなく固まる・遅い・Input/output error	7 位（通信）

■「起動すると出る」ときの見分け方

・新しく起動したタスク自身で出る → 起動直後に「他のタスクのロールオーバ」か「片付け」が新タスクのファイルを消した（1・2 位）。新タスクの根元が Stale になることは、起動後に根元を作り直さない限り無い。

・新しいタスクの起動に合わせて古いタスクで出る → 新タスクの起動時ロールオーバ（OnStartupTriggeringPolicy・時刻ポリシーの初回・rotate-on-boot）か、起動スクリプトの削除・作り直し（1・3 位）。

・新しいタスクは正常で、古いタスクだけ全部止まる → 根元の作り直し（4 位）。アクセスポイントが新しいマウントで根元を作り直している。

09　緊急対応：今すぐ直す

順番が大事。先に「消している側」を止めないと、直してもまた消される。入れ替えは新旧が重なるので、共有パスのままでは入れ替えの最中に再発しうる。

1　範囲をつかむ（10 分）

07-1 の Logs Insights で、Stale を出しているタスク・コンテナ・最初の時刻を一覧にする。1 タスクだけか、全タスクか、交互か。

▼

2　証拠を残す（10 分）

入れ替える前に、影響タスクで estale-diag.sh（または貼り付け用）の出力を保存する。ECS のサービスイベント・タスク一覧（startedAt・stoppedAt）も控える。入れ替えると FD の状態は消える。

▼

3　消している側を止める（即時）

片付けバッチのスケジュールを無効にする。EFS 上の手作業を止める（周知）。次のデプロイ・スケール操作を保留する。原因が 1 位なら、次のロールオーバ時刻（0 時・毎時 0 分）をまたぐ入れ替えを避ける。

▼

4　直す（ファイル単位：stat /mnt/logs は OK）

JBoss を reload すると、アプリとログの設定が作り直され、ファイルを新しく開く（タスクはそのまま）。確実なのはタスクの入れ替え。1 タスクずつ、ロールオーバ時刻を避けて行う。

▼

5　直す（根元：stat /mnt/logs が Stale）

そのタスクでは直らない（Fargate はコンテナから再マウントできない）。タスクを入れ替える。同じ根元を使う全タスクが対象。先に根元が正しく存在するか（新しいタスクがマウントできるか）を確かめる。

▼

6　確かめる

Logs Insights で Stale が止まったこと、EFS 上の app.log が伸びていること、estale-diag.sh が「異常なし」になったこと。

▼

7　その日のうちに

ログが失われないよう、コンソール（CloudWatch Logs）にも出す（Log4j2 の Console アペンダ／Failover、JBoss の CONSOLE）。原因の特定（08_切り分け）と、恒久対策（10）の計画に入る。

JBoss だけ再起動して開き直す（タスクはそのまま。ECS Exec で）

```
aws ecs execute-command --cluster <クラスター> --task <タスクID> --container <コンテナ名> --interactive ¥
  --command "/opt/jboss-eap/bin/jboss-cli.sh --connect --command=:reload"
# reload はアプリを配備し直すので、アプリに同梱した Log4j2 も新しくファイルを開く（数十秒〜数分、リクエストは受けられない）
# 根元が Stale のタスクでは直らない（タスクを入れ替える）
# readonlyRootFilesystem=true のタスクは ECS Exec が非対応（AWS 公式）。入れなければ stop-task で入れ替える
```

タスクを入れ替える

```
# 1 タスクずつ（サービスが代わりのタスクを起動する）
aws ecs stop-task --cluster <クラスター> --task <タスクID> --reason "ESTALE recovery"
# サービス全体を入れ替える（新旧が重なるので、共有パスのままならロールオーバ時刻を避ける）
aws ecs update-service --cluster <クラスター> --service <サービス> --force-new-deployment
```

状況	reload で直るか	入れ替えで直るか	注意
開いているファイルだけ Stale（根元は OK）	直る（多くの場合）	直る	消した側を止めてから。止めないと再発
ログのディレクトリが作り直された	直る（新しいディレクトリで開き直す）	直る	作り直した手順を止める
根元（/mnt/logs）が Stale	直らない	直る（唯一の方法）	根元が存在するか先に確認
通信の問題（固まる）	直らない	直ることが多い	SG・マウントターゲットを先に直す

■ 注意　共有パス（1 位）のまま force-new-deployment すると、入れ替えの重なりでまた相手のファイルを消しうる。緊急時は 1 タスクずつ stop-task し、ロールオーバ時刻をまたがないようにする。根本的には 10_恒久対策 P1 を先に入れる。

10　恒久対策：根本から起きないようにする

原則は 1 つ：「EFS 上の 1 つのファイルを書くのは 1 タスクだけ。消すのは止まったタスクのものだけ」。それに「根元を守る」「ログの正本は CloudWatch Logs」「監視」を足す。

■ 対策の一覧（優先順）

番号	対策	効く原因	効果	工数
P1	アプリログをタスクごとのディレクトリへ（1 ファイル 1 書き手）	1・5	根治（共有を無くす）	小（リンクと起動スクリプト）
P2	片付けは「止まったタスクのものだけ」。Log4j2 の Delete は自分のディレクトリだけ	2	根治	小
P3	起動スクリプト・手順から EFS 上の rm -rf・mv・作り直しを無くす	3	根治	小
P4	根元を守る（アクセスポイントで作る・消せる人を絞る・読み取り専用で調査）	4・6	予防	小〜中
P5	ログの正本を CloudWatch Logs（標準出力）に。EFS は控え、または廃止	全部	影響の最小化	中
P6	Stale の監視（メトリクスフィルター＋アラーム）と定期点検	全部	早期発見	小
P7	Log4j2 の保険（Failover で Console へ逃がす）	全部	ログを失わない	小
P8	デプロイ方式との組み合わせを決める（ローリング・ブルー/グリーンの重なり）	1	再発防止	小

■ P1　アプリログをタスクごとのディレクトリへ（推奨：リンク方式・アプリ変更なし）

JBoss のログですでに使っている「二段リンク」を、アプリログにも使う。イメージのリンクの向き先をタスクローカルの書き込み可能領域（standalone/tmp）に変え、起動時にそこから EFS のタスクごとのディレクトリへリンクを張る。readonlyRootFilesystem=true のまま使える。

いま（共有）	変えたあと（タスクごと）
/webapp/webapp9mf02/logs	/webapp/webapp9mf02/logs → standalone/tmp/app-log-target（イメージ）
→ /mnt/logs/front/inter-api/logs ← 全タスク共通	→ /mnt/logs/…/logs/<サービス>/app/<タスクごとの名前>（起動時）
全タスクが同じ app.log を開く	1 つのファイルを書くのは 1 タスクだけ
どれかのロールオーバが他のタスクのファイルを消す	ロールオーバ・Delete は自分のディレクトリの中だけ

Dockerfile（front。back は webapp9mb02）— impl/Dockerfile.front.snippet

```
RUN rm -rf /webapp/webapp9mf02/logs ¥
    && ln -s /opt/jboss-eap/standalone/tmp/app-log-target /webapp/webapp9mf02/logs
COPY app-log-dir.sh /usr/local/bin/app-log-dir.sh
```

efs-entrypoint.sh に追記（JBoss を exec する前。LOG_ID を決めた後なら JBoss ログと同じ名前になる）

```
. /usr/local/bin/app-log-dir.sh
setup_app_log_dir || die "アプリログのディレクトリを用意できません"
# → /mnt/logs/<コンポーネント>/logs/<サービス>/app/<LOG_ID>/ を作り、.owner（TaskARN 等）を書き、
#   standalone/tmp/app-log-target をそこへ向ける。APP_LOG_ID・APP_LOG_DIR を export
```

・アプリの log4j2.xml は /webapp/webapp9mf02/logs/app.log のままでよい（リンクの先が変わるだけ）。

・タスクごとの名前は、LOG_ID（起動時刻-乱数、または LOG_ID_MODE=taskid でタスク ID）を使う。同じタスクでコンテナだけ再起動したときは同じディレクトリを再利用する。

・.owner（TaskARN・コンテナ名・起動時刻・ホスト名）を書くので、片付け（P2）が「動いているタスクのものか」を判定できる。

・standalone/tmp にタスクローカルの書き込み可能ボリュームを当てていること（いまの設計どおり）が前提。

・移行期間：古い共有ディレクトリ直下のファイルは、全タスクが新しいパスに移ってから消す（移行中に消すと旧タスクが Stale）。

・集約・閲覧の仕組み（踏み台の tail、Fluent Bit 等）のパスを */app/*/app.log に変える。

▸ **別案：Log4j2 の設定でタスクごとに分ける（アプリの変更が必要）**

log4j2.xml の要点 ― impl/log4j2-pertask.xml

```
<Property name="APP_LOG_ID">${env:APP_LOG_ID:-${hostName}}</Property>
<Property name="LOG_DIR">/webapp/webapp9mf02/logs/${APP_LOG_ID}</Property>
<RollingFile name="APP_FILE" fileName="${LOG_DIR}/app.log" filePattern="${LOG_DIR}/app-%d{yyyy-MM-dd}.log.gz" ignoreExceptions="false">
  <DefaultRolloverStrategy><Delete basePath="${LOG_DIR}" maxDepth="1"> … </Delete></DefaultRolloverStrategy>
```

■ P2 片付けは「止まったタスクのものだけ」

同梱の tools/efs-log-housekeeping.sh は、削除の前に ECS へ生きているタスクを問い合わせ、次のどれかに当てはまるディレクトリは必ず残す。

残す条件	理由
持ち主のタスクが RUNNING、または停止処理中・停止直後（desired=STOPPED の一覧に載っている）	まだ書いている（ドレイン中も書く）
中のファイルが 24 時間以内に更新されている（--min-age-hours）	時計や判定のずれへの保険
.nfsXXXX がある	同じホストでまだ開いているプロセスがある
シンボリックリンク（current 等）と、リンクが指しているディレクトリ	案内札と、その先は残す
保存期間内（持ち主が分かる：--keep-days、分からない：--unknown-keep-days）	持ち主が分からないものは長めに残す
クラスターの RUNNING が 0 件なら中止（--allow-no-running で解除）	クラスター名の誤りで全部を古いと判定する事故を防ぐ

使い方（EFS をマウントした EC2・RHEL 9 系）

```
./tools/efs-log-housekeeping.sh -N -c <クラスター> -b /mnt/efs/logs-root/<コンポーネント>/logs/<サービス>/mid -b /mnt/efs/logs-root/<コンポーネント>/logs/<サービス>/app

# 判定を確かめたら -N を外し、cron では -y を付ける（既定は確認してから削除）
```

・Log4j2 の Delete は basePath を「自分のディレクトリ」にする。共有の親を指定すると他タスクのファイルを消す。

・共有ディレクトリ直下のファイルを日数で find -delete しない。

■ P3 起動スクリプト・手順から「消して作り直す」を無くす

・EFS 上では mkdir -p（あれば何もしない）だけを使う。rm -rf・mv・「空にしてから」をしない。

・Ansible・Terraform・手順書の「state: absent → directory」のような作り直しを点検する。

・レビューの観点：「EFS 上のパスに rm／mv／truncate／gzip をしていないか」。

■ P4 根元を守る

・根元はアクセスポイント（Path・作成時の所有者 uid/gid・権限）で作る。無ければマウント時に作られるので、手で作り直す必要がない。

・踏み台から EFS のルートを書き込みでマウントできる人・ロールを IAM（elasticfilesystem:ClientWrite・ClientRootAccess）とファイルシステムポリシーで絞る。

・調査用には読み取り専用のマウント（ClientMount のみの権限）を用意する。

・アクセスポイント・rootDirectory を変えるときは、使っているサービスを 0 にしてから、または新しいパスへ切り替えてから。

■ P5 ログの正本を CloudWatch Logs に

・JBoss の CONSOLE ハンドラ（JSON 形式も可）、Log4j2 の Console アペンダ（JsonTemplateLayout）で標準出力へ。awslogs または FireLens（Fluent Bit）で送る。

・EFS のファイルは「控え」と位置づける（失われても調査できる）。いずれは EFS へのログ出力をやめるのが最も単純（12-Factor のログ）。

・awslogs は mode=non-blocking と max-buffer-size でアプリを止めない設定にする。

■ P6 監視と定期点検

CloudWatch Logs のメトリクスフィルター＋アラーム（Terraform）— impl/terraform/main.tf

```
resource "aws_cloudwatch_log_metric_filter" "stale_file_handle" {
  for_each       = toset(var.log_group_names)
  log_group_name = each.value
  pattern        = "¥"Stale file handle¥""     # 大文字小文字を区別する完全一致の語句
  metric_transformation { name = "StaleFileHandleCount"  namespace = "Custom/EFSLogs"  value = "1"  default_value = "0" }
}
```

```
# アラーム: Sum >= 1（5 分）で通知
```

・CloudWatch Logs の正規表現フィルター（%…%）では ( ) と (?i) が使えない。完全一致の語句 "Stale file handle" を使う。

・週に 1 回：全タスクで estale-diag.sh -q（または Logs Insights で Stale 0 件）、.nfs ファイルの有無、mid/・app/ のディレクトリ数。

■ P7 Log4j2 の保険（書けなくてもログを失わない）

・Failover アペンダ：ファイル（ignoreExceptions="false"）に書けないときは Console（CloudWatch Logs）へ。retryIntervalSeconds ごとにファイルを試し直す。実機 S10：出力 125 行のうち 80 行が標準出力へ。

・Log4j2 は Stale になったファイルを自分で開き直さない。保険はあくまで「失わない」ためで、根本対策（P1）の代わりにはならない。

・Log4j 2.25.0 以降（ロールオーバ失敗時の開き直しの修正）、2.25.5／2.26.1 以降（createOnDemand の修正）を使う。

■ P8 デプロイ方式との組み合わせ

・ローリング（最小 100%・最大 200%）、ブルー/グリーン（2025-07～）、リニア・カナリア（2025-10～）は、新旧が同時に動く時間がある。共有パスのままだと 1 位が増える。

・P1 を入れた後なら、どの方式でも Stale は起きない（実機 S7）。P1 の前は、デプロイをロールオーバ時刻から離す。

・（参考）タスクごとの EBS ボリューム（2024-01～）は他タスクと共有しないので「他人が消す」は起きないが、タスクとともに消える前提。ログの正本にはしない。

■ 検証 S7：タスクごとのディレクトリにすると、最も厳しい NFSv3 でも Stale A=0 B=0 / 出力 A=225 B=225 / 残った A=225 B=225。

11　運用回避：起こさないための約束ごと

恒久対策が入るまでのつなぎ、そして入った後も守る約束。EFS は「みんなの倉庫」なので、自分の物以外を捨てない・使っている物を捨てない。

■ EFS 上でやってはいけない操作

操作	なぜだめか	代わりにどうする
動いているタスクのログを手で消す・圧縮する・移動する（rm・gzip・mv）	開いている側が Stale になる（ローカルとちがう）	止まったタスクのものだけ。どうしても必要ならそのタスクを先に止める
ログのディレクトリやその上を消して作り直す（rm -rf → mkdir、mv → mkdir）	名前が同じでも実体が変わり、全員が Stale	新しいパスを作って切り替え、古い方は最後に消す
根元（rootDirectory・アクセスポイントのルート）を消す・作り直す、アクセスポイントを作り直す	そのタスクは入れ替えるまで全滅	使っているサービスを 0 にしてから
共有ディレクトリを日数で find -delete する	長く動くタスク・めったに書かないログを消す	efs-log-housekeeping.sh（止まったタスクだけ）
DataSync・rsync --delete でログ置き場を同期先にする	同期の削除が使用中のファイルを消す	ログ置き場を同期の削除対象にしない
.nfsXXXX を消す	消せない（使用中）か、別の .nfs になるだけ	持っているプロセス（タスク）を止めると自然に消える

■ 片付けのルール

・消すのは「保存期間を過ぎた」かつ「持ち主のタスクが止まっている」ものだけ（ECS に問い合わせて確かめる）。

・.nfs がある・24 時間以内に更新がある・リンクが指している、は残す。

・RUNNING のタスクが 0 件と出たら止める（クラスター名の誤りかもしれない）。

・デプロイ・スケールの最中は片付けない（実行時刻をずらす）。

・まず -N（判定だけ）で確かめてから実行する。結果（--report の TSV）を残す。

■ 構成を変えるときの手順（切り替えてから消す）

1　新しいパスを作る

mkdir -p で新しいディレクトリ（またはアクセスポイント）を用意する。古いパスは触らない。

▼

2　タスクを新しいパスへ切り替える

イメージ・タスク定義を変えてデプロイ。新しいタスクは新しいパスに書く。

▼

3　全タスクが新しいパスになったのを確かめる

旧タスクがすべて STOPPED になり、停止から十分時間がたったこと（describe-tasks、estale-diag の FD 一覧）。

▼

4　古いパスを消す

最後に消す。消す前に .nfs が無いことを確かめる。

■ 共有パスが残っている間の暫定の約束（P1 まで）

・デプロイはロールオーバ時刻（0 時・毎時 0 分）から 20 分以上離す。

・起動時ロールオーバ（OnStartupTriggeringPolicy・rotate-on-boot）を外す。

・Stale のアラームが出たら 09_緊急対応の手順で 1 タスクずつ直す。

・（可能なら）一時的に desiredCount を 1 にして重なりを減らす。

■ チェックリスト（○ / × / – を選べる）

チェック	確認すること
	アプリログのリンク先は、タスクごとのディレクトリになっているか
	log4j2.xml の Delete の basePath は自分のディレクトリだけか
	起動時ロールオーバ（OnStartupTriggeringPolicy・rotate-on-boot）は共有パスで使っていないか
	片付けは止まったタスクだけを消す仕組みか（ECS に問い合わせているか）
	起動スクリプト・手順に EFS 上の rm -rf・mv・作り直しが無いか
	根元はアクセスポイントで作り、消せる人を絞っているか
	ログの正本は CloudWatch Logs に出ているか
	Stale file handle のアラームがあるか
	front と back、サービス間で同じディレクトリを共有していないか
	デプロイ中に片付けが動かないか

12　コマンド集

< > の中を置き換えて使う。AWS CLI は aws login --remote 済みの端末で。

CloudWatch Logs Insights

```
fields @timestamp, @logStream, @message | filter @message like /Stale file handle/ | stats count(*) as n, min(@timestamp) as first, max(@timestamp) as last by @logStream | sort first asc

fields @timestamp, @logStream, @message | filter @message like /Stale file handle|Unable to write to stream|Unable to close stream|FileNotFoundException/ | sort @timestamp asc | limit 500
```

ECS：タスク・デプロイ・重なり

```
aws ecs list-tasks --cluster <C> --service-name <S> --desired-status RUNNING --query 'taskArns' --output text
aws ecs list-tasks --cluster <C> --service-name <S> --desired-status STOPPED --query 'taskArns' --output text

aws ecs describe-tasks --cluster <C> --tasks <ID...> --query 'tasks[].[taskArn,lastStatus,startedAt,stoppingAt,stoppedAt,stoppedReason]' --output table

aws ecs describe-services --cluster <C> --services <S> --query 'services[0].events[:30].[createdAt,message]' --output text

aws ecs describe-services --cluster <C> --services <S> --query 'services[0].[desiredCount,deploymentConfiguration,deployments[].[status,rolloutState,createdAt]]'

aws ecs describe-task-definition --task-definition <family:rev> --query 'taskDefinition.volumes'
```

ECS Exec：コンテナの中

```
aws ecs execute-command --cluster <C> --task <ID> --container <name> --interactive --command "/bin/bash"
stat /mnt/logs ; ls -la /webapp/*/logs/ ; readlink -f /webapp/*/logs
for P in /proc/[0-9]*; do case "$(readlink $P/exe 2>/dev/null)" in */java) echo ${P#/proc/};; esac; done
ls -l /proc/<pid>/fd | grep /mnt/logs
(: < /proc/<pid>/fd/<n>) && echo OK      # NG ならエラー文（Stale file handle 等）が出る
grep -A9 'mounted on /mnt/logs ' /proc/self/mountstats
find -L /webapp/*/logs -maxdepth 3 -name '.nfs*'
bash estale-diag.sh [-q] [--probe]
```

復旧

```
aws ecs execute-command --cluster <C> --task <ID> --container <name> --interactive --command "/opt/jboss-eap/bin/jboss-cli.sh --connect --command=:reload"

aws ecs stop-task --cluster <C> --task <ID> --reason "ESTALE recovery"
aws ecs update-service --cluster <C> --service <S> --force-new-deployment
```

EFS・CloudTrail・メトリクス

```
aws efs describe-access-points --file-system-id <fs-ID> --query 'AccessPoints[].[AccessPointId,RootDirectory.Path,LifeCycleState]' --output table
```

```
aws efs describe-mount-targets --file-system-id <fs-ID> --query 'MountTargets[].[MountTargetId,AvailabilityZoneName,IpAddress,LifeCycleState]' --output table

aws cloudtrail lookup-events --start-time <UTC> --end-time <UTC> --lookup-attributes AttributeKey=EventSource,AttributeValue=elasticfilesystem.amazonaws.com --query 'Events[].[EventTime,EventName,Username]' --output table
aws cloudwatch get-metric-statistics --namespace AWS/EFS --metric-name ClientConnections --dimensions Name=FileSystemId,Value=<fs-ID> --start-time <開始> --end-time <終了> --period 300 --statistics Sum
```

片付け（同梱ツール）

```
./tools/efs-log-housekeeping.sh -N -c <C> -b <EFS 上の mid/ や app/ の親> --report /tmp/hk.tsv
./tools/efs-log-housekeeping.sh -y -c <C> -b <…> -k 30 --unknown-keep-days 90 --min-age-hours 24
```

13　実装：同梱のスクリプト・設定とテスト

すべて Log_Stale_Handle フォルダにある。運用シェルは既存の規約（common.sh・aws login --remote の確認・スイッチバック・dry-run）に合わせた。

■ ファイル一覧

ファイル	役割	動く場所
tools/estale-diag.sh	コンテナの中で、根元・リンク・Java の FD（STALE/REPLACED/MISSING/SILLY/OK）・.nfs・再接続を調べて判定（終了コード 0/1/2）	コンテナ（ECS Exec）
tools/efs-log-housekeeping.sh	EFS 上のタスクごとのディレクトリを、止まったタスクのものだけ片付ける（ECS に問い合わせ）	EFS をマウントした EC2（RHEL 9 系）
tools/common.sh	運用シェルの共通部品（ログ・die・run・confirm・AWS エラー分類）	ー
impl/app-log-dir.sh	アプリログをタスクごとのディレクトリへ向ける（.owner も書く）。entrypoint から source	コンテナの起動時
impl/Dockerfile.front.snippet	アプリログのリンクを standalone/tmp/app-log-target へ向ける変更例	イメージのビルド
impl/log4j2-pertask.xml	Log4j2 側でタスクごとに分ける例（Failover・自分のディレクトリだけの Delete）	アプリ（WAR）
impl/terraform/main.tf	Stale file handle のメトリクスフィルター＋アラーム	Terraform
lab/run_lab.sh ほか	Docker で NFS サーバとクライアント 2 台を立てて再現する検証一式（14_検証結果）	Docker
tests/run_tests.sh	片付け・診断・起動スクリプトのテスト（偽の aws コマンドで AWS なしに実行）	UBI 9.8 コンテナ

■ estale-diag.sh

使い方と判定

```
bash estale-diag.sh          # NFS マウントを自動検出、/webapp/*/logs 等のリンク、java プロセスの FD を全部
bash estale-diag.sh -q --probe # 判定だけ表示＋書き込みテスト
bash estale-diag.sh -m /mnt/logs -l /webapp/webapp9mf02/logs -p <pid>
# 終了コード 0=異常なし / 1=ファイル単位の異常 / 2=根元が Stale・応答なし / 3=引数の誤り
# FD の判定: STALE（サーバ上で消えた）/ MISSING（名前が無い）/ REPLACED（名前は別の実体）/ SILLY（.nfs）/ OK
```

・ps・pgrep・lsof・jq を使わず、/proc と coreutils・findutils だけで動く（UBI の最小構成でも可）。

・確認はすべて timeout -s KILL 付き（応答しない hard マウントで固まらない）。

・stat だけでなく読み取りで開き直してサーバに確かめる（NFS の属性キャッシュにだまされない）。

■ efs-log-housekeeping.sh

オプション	意味
-b, --base DIR（必須・複数可）	タスクごとのディレクトリが並ぶ親（mid/・app/ など）
-c, --cluster 名前（必須・複数可）	生きているタスクを問い合わせるクラスター
-k, --keep-days（既定 30）／--unknown-keep-days（既定 90）	保存日数（持ち主が分かる／分からない）
--min-age-hours（既定 24）	この時間内に更新があれば必ず残す
--allow-no-running	RUNNING 0 件でも続ける（既定は中止）
-N, --dry-run／-y, --yes／--report FILE	判定だけ／確認なしで削除／判定を TSV で保存
-r, -p, -A, -S	リージョン・プロファイル・自動スイッチバック・スイッチバック用シェル（規約どおり）

終了コード：0 正常 / 2 引数の誤り / 3 一部を消せなかった / 4 安全に判定できない（RUNNING 0 件・クラスターが無い）/ 10 未認証 / 11 ECS の参照権限なし / 12 スイッチバックの失敗。必要な IAM は ecs:ListTasks と sts:GetCallerIdentity。

■ テスト

tests/run_tests.sh を UBI 9.8 のコンテナで実行（偽の aws コマンドで、判定・削除・認証・権限・スイッチバック・起動スクリプトを確認）。結果は 59 件すべて合格。shellcheck（koalaman/shellcheck:stable）も全スクリプトで指摘 0。Terraform は aws プロバイダ 6.64.0 で validate・fmt を確認。

テストの実行（Windows の Git Bash から）

```
MSYS_NO_PATHCONV=1 docker run --rm -v "$(cygpath -m "$PWD"):/work:ro" registry.access.redhat.com/ubi9/ubi:9.8 bash /work/tests/run_tests.sh /work
MSYS_NO_PATHCONV=1 docker run --rm -v "$(cygpath -m "$PWD"):/w" -w /w koalaman/shellcheck:stable -x tools/*.sh impl/app-log-dir.sh lab/run_lab.sh
bash lab/run_lab.sh          # 再現実験（約 12 分。--privileged が必要。bash lab/run_lab.sh S6 S7 で指定ケースだけ）
```

14　検証結果：Docker で NFS を立てて再現した

EFS の代わりに Linux カーネルの NFS サーバ（knfsd）を 1 台、ECS タスクの代わりに UBI 9.8＋OpenJDK 21＋Log4j2 のコンテナを 2 台（タスク A・B）立て、ご質問と同じリンク（/webapp/webapp9mf02/logs → /mnt/logs/front/inter-api/logs）で確かめた。再実行は bash lab/run_lab.sh。

■ 環境

・date=2026-10-09 23:30:56 +0900

・docker=29.8.1 kernel=5.15.153.1-microsoft-standard-WSL2

・client=Red Hat Enterprise Linux release 9.8 (Plow) java=openjdk version "21.0.12.1" 2026-08-18 LTS log4j=2.26.1

・server=Debian 12.15 knfsd（Docker の VM のカーネルに内蔵） nfs-kernel-server=1:2.6.2-4+deb12u1

・マウントは EFS と同じ NFSv4.1・hard・timeo=600・retrans=2・noresvport。比較のため状態を持たない NFSv3 も使った。

・タスク A と B は別のコンテナ（別のネットワーク名前空間）＝別の NFS クライアント。ECS で別タスクになるのと同じ関係。

・ロールオーバは本番の 1 時間・1 日を 10 秒に縮めた（TimeBasedTriggeringPolicy interval=10、gz 圧縮）。

・サーバの猶予期間（NFSv4 grace）を 10 秒に縮め、明けてから始めた（起動直後は新しい OPEN が待たされるため）。

■ 結果の一覧

ケース	NFS	条件	期待	観測	判定
S1_v41	v4.1	別クライアントが開いているファイルを削除	knfsd は実体を残すのでエラーにならない（書いた行は消える）	SUMMARY writes=74 errors=0 firstError=-	OK
S1_v3	v3	別クライアントが開いているファイルを削除	書き込みが ESTALE（IOException: Stale file handle）	SUMMARY writes=74 errors=47 firstError=14:31:37.951 seq=28	OK
S2	v4.1	ログのディレクトリを作り直す	掴んでいるもの（カレントディレクトリ）は ESTALE、パス名で開き直すものは自動で直る	cwd=ESTALE / パス名=OK / 新規作成=OK	OK
S3_v41	v4.1	マウントの根元を作り直す	ls・stat・mkdir が ESTALE、マウントし直すまで直らない	マウント中は ESTALE / 再マウントで回復 / Java: SUMMARY writes=67 errors=0 firstError=-	OK
S3_v3	v3	マウントの根元を作り直す	ls・stat・mkdir が ESTALE、マウントし直すまで直らない	マウント中は ESTALE / 再マウントで回復 / Java: SUMMARY writes=67 errors=57 firstError=14:32:39.388 seq=11	OK
S4	v4.1	同じクライアントで使用中ファイルを削除	.nfsXXXX に改名されエラーにならない。閉じると中身ごと消える	SUMMARY writes=47 errors=0 firstError=- / 閉じた後の .nfs=0	OK

S5	v4.1	書き込み中にマウントを外して付け直す（仮説）	ESTALE にならない（古い FD は外したマウント経由で書き続ける）	SUMMARY writes=67 errors=0 firstError=- / サーバ上の行数=67	OK
S6_v3	v3	2 タスクが同じファイル＋ロールオーバ	片方に AppenderLoggingException … Caused by: IOException: Stale file handle	Stale の出現 A=12 B=10回 / 出力 A=225 B=225 / 残った A=61 B=103	OK
S6_v41	v4.1	2 タスクが同じファイル＋ロールオーバ	knfsd ではエラーが出ないことがあるが、行は欠ける	Stale の出現 A=0 B=0回 / 出力 A=225 B=225 / 残った A=49 B=175	OK
S7	v3	タスクごとのディレクトリに書く（対策後）	ESTALE 0 件・欠けた行 0	Stale A=0 B=0 / 出力 A=225 B=225 / 残った A=225 B=225	OK
S8_v3	v3	診断スクリプトの判定	正常は 0、改名後は REPLACED（1）。削除後は v3 で STALE（1）、knfsd v4.1 は STALE か REPLACED（1）	正常=exit=0 / 改名+作り直し 1 秒後=exit=0（31 秒後=exit=1）REPLACED 表示 1 / 改名先も削除=exit=1（STALE 表示 1）	OK
S8_v41	v4.1	診断スクリプトの判定	正常は 0、改名後は REPLACED（1）。削除後は v3 で STALE（1）、knfsd v4.1 は STALE か REPLACED（1）	正常=exit=0 / 改名+作り直し 1 秒後=exit=1（31 秒後=-）REPLACED 表示 1 / 改名先も削除=exit=1（STALE 表示 1）	OK
S9	v3	サーバ再起動 / fsid の付け替え	再起動では ESTALE にならず、付け替えると ESTALE	再起動=OK / 付け替え=ESTALE	OK
S10	v3	Failover の保険（ファイルが消された後）	ファイルに書けない間のログが標準出力（CloudWatch Logs）に出る	出力 125 行のうち 80 行が標準出力へ	OK

■ 読み取れること

・別クライアントが開いているファイルを消すと、状態を持たない NFSv3 では書き込みが Stale になる（S1 v3）。EFS も同じく Stale を返す（AWS 技術者の報告とご質問の事象）。

・Linux の knfsd（NFSv4.1）は開いているあいだ実体を残すのでエラーにならないが、書いた行は名前の無いファイルに入って消える（S1 v4.1・S6 v4.1）。「エラーが出ない＝安全」ではなく、「手元の NFS では起きない＝EFS でも起きない」でもない。

・ご質問と同じ形（AppenderLoggingException … Caused by: java.io.IOException: Stale file handle）は、2 タスク共有＋ロールオーバで再現した（S6 v3）。

・タスクごとのディレクトリにすると、同じ条件で Stale 0 件・欠けた行 0（S7）。

・Log4j2 の Failover（impl/log4j2-pertask.xml）は、ファイルを消されて書けなくなった後のログを標準出力へ逃がした（S10）。

・マウントの付け直し（仮説）では Stale にならない（S5）。根元の作り直しでは全操作が Stale になり、マウントし直すまで直らない（S3）。

・同じクライアント内の削除は .nfs になりエラーにならない（S4）。サーバの再起動（同じ fsid）では Stale にならず、エクスポートの付け替え（fsid 変更）で Stale（S9）。

・診断スクリプトは、正常（0）・REPLACED（1）・削除後の STALE（1）・根元の Stale（2）・.nfs（1）を正しく判定した（S1・S3・S4・S8）。ただし NFSv3 では、他のクライアントの改名に気づくまでディレクトリの属性キャッシュ（既定 30 秒）ぶん遅れる（S8 v3：1 秒後は見逃し、31 秒後に検出）。EFS の既定の v4.1 ではすぐ気づく。

■ 抜粋：ご質問と同じ例外（S6 v3：2 タスクが同じ app.log･10 秒ごとのロールオーバ）

タスク A／B の標準エラー（Log4j2 の StatusLogger）

```
$ [a] grep -n -m6 -E 'ERROR|Caused by|IOException' /tmp/s6.out; grep -o 'SUMMARY.*' /tmp/s6.out

2:2026-10-09T14:34:00.195889991Z main ERROR RollingFileManager /webapp/webapp9mf02/logs/app.log Unable to close stream: java.io.IOException: Stale file handle

33:2026-10-09T14:34:13.050316840Z main ERROR Unable to write to stream /webapp/webapp9mf02/logs/app.log for appender APP
62:Caused by: java.io.IOException: Stale file handle
68:2026-10-09T14:34:13.051380536Z main ERROR An exception occurred processing Appender APP
97:Caused by: java.io.IOException: Stale file handle
SUMMARY events=225
$ [b] grep -n -m6 -E 'ERROR|Caused by|IOException' /tmp/s6.out; grep -o 'SUMMARY.*' /tmp/s6.out
2:2026-10-09T14:34:03.067980500Z main ERROR Unable to write to stream /webapp/webapp9mf02/logs/app.log for appender APP
31:Caused by: java.io.IOException: Stale file handle
```

■ 抜粋：別クライアントによる削除（S1 v3）

Java（FileOutputStream で追記）の出力

```
removed '/mnt/logs/front/inter-api/logs/app.log'
14:31:37.958 [A] write FAILED at seq=28
java.io.IOException: Stale file handle
    at java.base/java.io.FileOutputStream.writeBytes(Native Method)
    at java.base/java.io.FileOutputStream.write(FileOutputStream.java:346)
14:31:43.666 [A] still failing: errors=20 (Stale file handle)
14:31:49.684 [A] still failing: errors=40 (Stale file handle)
14:31:52.089 [A] close FAILED: java.io.IOException: Stale file handle
14:31:52.091 [A] SUMMARY writes=74 errors=47 firstError=14:31:37.951 seq=28
>>> 判定 [OK] 期待: 書き込みが ESTALE（IOException: Stale file handle） / 観測: SUMMARY writes=74 errors=47 firstError=14:31:37.951 seq=28
```

■ 抜粋：根元の作り直し（S3 v4.1）

コンテナの中の操作と診断スクリプト

```
$ [b] stat -c 'before inode=%i' /mnt/logs/logs-root; rm -rf /mnt/logs/logs-root && mkdir -p /mnt/logs/logs-root/front/inter-api/logs && stat -c 'after  inode=%i' /mnt/logs/logs-root

before inode=138237
after  inode=138266

$ [a] ls -la /mnt/logs; echo rc=$?; stat /mnt/logs; echo rc=$?; mkdir -p /mnt/logs/front/inter-api/logs; echo rc=$?; touch /webapp/webapp9mf02/logs/probe.txt; echo rc=$?

ls: cannot open directory '/mnt/logs': Stale file handle
```

rc=2
stat: cannot statx '/mnt/logs': Stale file handle
rc=1
mkdir: cannot create directory '/mnt/logs': Stale file handle
rc=1
touch: cannot touch '/webapp/webapp9mf02/logs/probe.txt': No such file or directory
rc=1
$ [a] bash /opt/tools/estale-diag.sh -q --probe; echo exit=$?

- /mnt/logs の根元が Stale: EFS 側で rootDirectory／アクセスポイントのルート（またはその上）が削除・作り直しされた。このタスクでは直らない → タスクを入れ替える

- /webapp/webapp9mf02/logs の行き先が無い: EFS 側のディレクトリが未作成・削除済み、または上位が Stale（マウント欄を確認）

- Java が掴んでいるファイル 1 個が Stale: 別タスク・別ホストがそのファイル（または親ディレクトリ）を削除した。JBoss の reload かタスク入れ替えで開き直す。根本対策はタスクごとのログパス

■ 抜粋：仮説の検証（S5：書き込み中にマウントを外して付け直す）

操作と結果

$ [a] umount /mnt/logs; echo rc=$?
umount: /mnt/logs: target is busy.
rc=32
$ [a] umount -l /mnt/logs; echo rc=$?; mount -i -t nfs4 -o vers=4.1,addr=172.30.0.10,hard,timeo=600,retrans=2,noresvport 172.30.0.10:/logs-root /mnt/logs; echo mount_rc=$?; grep ' /mnt/logs ' /proc/mounts | cut -c1-60; echo hi > /webapp/webapp9mf02/logs/after-remount.txt && echo NEW_OPEN_OK
rc=0
mount_rc=0
NEW_OPEN_OK
14:33:41.432 [A] SUMMARY writes=67 errors=0 firstError=-
$ [b] wc -l /mnt/logs/front/inter-api/logs/app.log; tail -1 /mnt/logs/front/inter-api/logs/app.log
67 /mnt/logs/front/inter-api/logs/app.log

■ 抜粋：同じクライアント内の削除（S4）と診断（S8）

S4：.nfs への改名

removed '/webapp/webapp9mf02/logs/app.log'
-rw-r--r-- 1 root root 234 Oct 9 14:33 .nfs0000000000021c1900000004
- FD 1 個が .nfs ファイル（同じタスク内で使用中に削除）：容量を使い続け、閉じると中身ごと消える
14:33:16.325 [A] SUMMARY writes=47 errors=0 firstError=-

>>> 判定 [OK] 期待: .nfsXXXX に改名されエラーにならない。閉じると中身ごと消える / 観測: SUMMARY writes=47 errors=0 firstError=- / 閉じた後の .nfs=0

S8：診断スクリプトの判定（v3）

```
# S8_v3  診断スクリプトの判定: 正常 → 改名して作り直し（REPLACED）→ 改名先も削除（STALE）（NFSv3）
$ [a] bash /opt/tools/estale-diag.sh; echo exit=$?
集計: STALE=0 REPLACED=0 MISSING=0 SILLY=0 OK=1
exit=0
$ [a] bash /opt/tools/estale-diag.sh -q; echo exit=$?
exit=0
$ [a] NFSv3 はディレクトリの属性キャッシュ（acdirmin=30 秒）が切れるまで改名に気づかないことがある。31 秒待ってもう一度
$ [a] bash /opt/tools/estale-diag.sh -q; echo exit=$?

- FD 1 個が古い実体を掴んでいる（名前は別ファイル）: 同じパスを複数タスクが使い、他タスクがローテーションした跡。ログが想定と違うファイルに入っている

exit=1
$ [a] bash /opt/tools/estale-diag.sh -q; echo exit=$?

- Java が掴んでいるファイル 1 個が Stale: 別タスク・別ホストがそのファイル（または親ディレクトリ）を削除した。JBoss の reload かタスク入れ替えで開き直す。根本対策はタスクごとのログパス

exit=1
>>> 判定 [OK] 期待: 正常は 0、改名後は REPLACED（1）。削除後は v3 で STALE（1）、knfsd v4.1 は STALE か REPLACED（1） / 観測: 正常=exit=0 / 改名+作り直し 1 秒後=exit=0（31 秒後=exit=1）
REPLACED 表示 1 / 改名先も削除=exit=1（STALE 表示 1）
```

■ 抜粋：Log4j2 の保険（S10：Failover）

ファイルが消された後、標準出力へ逃げた行

```
# S10  Log4j2 の保険（impl/log4j2-pertask.xml の Failover）: 別クライアントがファイルを消しても、ログは標準出力へ逃げる（NFSv3）
removed '/mnt/logs/front/inter-api/logs/task-a-0001/app.log'
$ [a] grep -m4 -E 'ERROR|Stale' /tmp/s10.out | cut -c1-200; echo '--- 標準出力へ逃げた行（Failover）'; grep -c 'lab - task=A seq=' /tmp/s10.out; grep 'lab - task=A seq=' /tmp/s10.out | tail -2; grep -o 'SUMMARY.*' /tmp/s10.out
2026-10-09T14:40:02.461299956Z main ERROR Unable to write to stream /webapp/webapp9mf02/logs/task-a-0001/app.log for appender APP_FILE
Caused by: java.io.IOException: Stale file handle
--- 標準出力へ逃げた行（Failover）
80
2026-10-09T14:40:18.103Z INFO  [main] lab - task=A seq=124
2026-10-09T14:40:18.304Z INFO  [main] lab - task=A seq=125
SUMMARY events=125
>>> 判定 [OK] 期待: ファイルに書けない間のログが標準出力（CloudWatch Logs）に出る / 観測: 出力 125 行のうち 80 行が標準出力へ
```

■ 限界（この検証で言えないこと）

・サーバは EFS そのものではない。EFS の「消されたら Stale」は、AWS 技術者の報告（2020）とご質問の事象から判断した。EFS 上での確認は 07-8 の手順で検証環境で行える。

・クライアントのカーネルは Docker Desktop（WSL2）の 5.15。Fargate の VM や EC2 のカーネルとは版が違うが、ESTALE・silly rename・retry_estale の仕組みは同じ。

・ECS のタスク・Fargate の監督コンテナ・efs-proxy（TLS）は使っていない（コンテナの中で直接マウントした）。
・欠けた行の数はタイミングで毎回変わる（観測の数字は 1 回分）。

15　用語集（小学生向けのたとえつき）

むずかしい言葉を、たとえと一緒に。

用語	意味	たとえ
NFS	ネットワークの向こうのファイルを、自分のディスクのように使う仕組み（1984 年〜）	遠くの共同倉庫
Amazon EFS	AWS の NFS サービス。たくさんのタスクから同時に使える	学校の共同倉庫
ファイルハンドル	NFS サーバが発行する「その実体」の証明	ロッカーの鍵・引換券
inode（アイノード）	ファイルの実体（中身と属性）。番号で区別	ノートそのもの
ESTALE（116）	Linux のエラー「Stale file handle」	「その鍵はもう古いよ」
NFS4ERR_STALE（70）	NFS サーバが返す「その引換券は無効」	倉庫の係の「そんなロッカーは無い」
FD（ファイルディスクリプタ）	プロセスが開いたファイルを指す番号	手に持っている整理券
silly rename（.nfsXXXX）	同じクライアントで使用中のファイルを消すとき、改名して閉じるまで残す工夫	忘れ物箱
close-to-open 一貫性	開くときにサーバへ確かめ、閉じるときに書き戻す NFS の約束	借りるときと返すときだけ確認
属性キャッシュ	ファイルの大きさや時刻を数秒〜1 分覚えておくこと	メモを見て答える（少し古いことがある）
マウント	倉庫の場所を自分のフォルダにつなぐこと	入館証
根元（rootDirectory・アクセスポイントのルート）	マウントのいちばん上	建物そのもの
アクセスポイント	EFS の入口。根元・ユーザーを決められる	その教室専用の入口
efs-proxy	EFS への暗号化通信（TLS）を中継する AWS 製の部品（efs-utils v2〜）	倉庫への専用の配達係
シンボリックリンク	別の場所を指す案内	道しるべ
ロールオーバ	ログファイルを時刻・サイズで切り替える（改名・圧縮・削除）	日記帳を新しくする
StatusLogger	Log4j2 が自分のエラーを出す先（標準エラー）	係の連絡帳
AppenderLoggingException	Log4j2 のアペンダが書けなかったときの例外	「書けませんでした」の報告
ECS タスク	コンテナのまとまり。タスクごとに別の NFS クライアント	教室
ローリングデプロイ	新しいタスクを起動してから古いタスクを止める入れ替え	新しい当番が来てから前の当番が帰る
ブルー/グリーン・カナリア・リニア	新旧 2 組を並べて少しずつ切り替える方式（ECS 組み込み 2025〜）	2 クラス合同の時間が長い

ECS Exec	動いているコンテナに入ってコマンドを実行する機能（2021〜）	教室をのぞく窓
retry_estale	パス名の操作が Stale のとき、名前を引き直して 1 回やり直す Linux の仕組み	道しるべを見直して行き直す
猶予期間（grace period）	NFSv4 サーバの起動直後、前の状態を取り戻すために新しい OPEN を待たせる時間	朝礼が終わるまで教室に入れない

16　参考資料

2026 年 10 月に確認したもの。Excel 版は URL のセルをクリックすると開く。

種別	資料	URL・場所
AWS	Use Amazon EFS volumes with Amazon ECS（Fargate 1.4.0 以上、監督コンテナ、アクセスポイント）	https://docs.aws.amazon.com/AmazonECS/latest/developerguide/efs-volumes.html
AWS	Enforcing a root directory with an access point（根元は次のマウントで作り直される）	https://docs.aws.amazon.com/efs/latest/ug/enforce-root-directory-access-point.html
AWS	Recommended NFS mount settings（noresvport・hard・timeo=600・retrans=2）	https://docs.aws.amazon.com/efs/latest/ug/mounting-fs-nfs-mount-settings.html
AWS	Troubleshooting file operation errors related to quotas（同時に開けるファイル数など）	https://docs.aws.amazon.com/efs/latest/ug/troubleshooting-efs-fileop-errors.html
AWS	Filter pattern syntax（CloudWatch Logs の正規表現は ( ) 不可・大文字小文字を区別）	https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/FilterAndPatternSyntax.html
AWS	Monitor Amazon ECS containers with ECS Exec（コマンドは root で動く・readonlyRootFilesystem は非対応）	https://docs.aws.amazon.com/AmazonECS/latest/developerguide/ecs-exec.html
AWS	AWS Fargate launches Platform Version 1.4（2020-04-08、EFS 対応）	https://aws.amazon.com/about-aws/whats-new/2020/04/aws-fargate-launches-platform-version-14
AWS	Introducing Amazon EFS（2015-04）	https://aws.amazon.com/about-aws/whats-new/2015/04/introducing-amazon-efs/
AWS	Amazon EFS is Now Generally Available（2016-06）	https://aws.amazon.com/about-aws/whats-new/2016/06/amazon-elastic-file-system-efs-is-now-generally-available
AWS	Amazon EFS increases maximum per-client throughput to 1.5 GiB/s（2024-05）	https://aws.amazon.com/about-aws/whats-new/2024/05/amazon-efs-maximum-per-client-throughput-1-5-gibs/
AWS	Amazon ECS now supports built-in Linear and Canary deployments（2025-10）	https://aws.amazon.com/about-aws/whats-new/2025/10/amazon-ecs-built-in-linear-canary-deployments
AWS	Choosing between Amazon ECS Blue/Green Native or AWS CodeDeploy（組み込みブルー/グリーン 2025-07）	https://aws.amazon.com/blogs/devops/choosing-between-amazon-ecs-blue-green-native-or-aws-codedeploy-in-aws-cdk
GitHub	aws/efs-utils（README・CHANGELOG-2.x／3.x：efs-proxy、watchdog）	https://github.com/aws/efs-utils
Linux	LKML 2020-11「NFS: Retry the CLOSE if the embedded GETATTR is rejected with ERR_STALE」（EFS で他クライアントが消したファイルに ESTALE）	https://lkml.iu.edu/2011.2/05183.html
Linux	同スレッド（Trond Myklebust の返信：STALE は致命的な通知）	https://lkml.iu.edu/2011.2/06552.html

Linux	vfs: add a retry_estale helper（Jeff Layton、2012）	https://lkml.iu.edu/hypermail/linux/kernel/1209.0/03099.html
Linux	LWN：ESTALE の再試行をパス名のシステムコールへ（3.7～3.8）	https://lwn.net/Articles/521686
Linux	errno.h: remove "NFS" from descriptions（glibc の文言変更に合わせた 2013 年の変更）	https://lkml.iu.edu/1310.3/02428.html
Linux	open(2)（O_APPEND は NFS で複数の書き手だと壊れうる）	https://man7.org/linux/man-pages/man2/open.2.html
書籍	Managing NFS and NIS, 2nd Ed.「Stale filehandles」（他のホストが消すと Stale、作り直しても直らない）	https://docstore.mik.ua/orelly/networking_2ndEd/nfs/ch18_08.htm
RFC	RFC 1094（NFS v2）／RFC 1813（v3）／RFC 7530（v4.0）／RFC 8881（v4.1）	https://www.rfc-editor.org/rfc/rfc8881
Apache	Log4j 2 Release Notes（2.25.0 ロールオーバ失敗時の開き直し、2.25.5／2.26.1 createOnDemand）	https://logging.apache.org/log4j/2.x/release-notes.html
Apache	LOG4J2-3099「Log4j2 do not recreate log file after deleting log on runtime」（Invalid）	https://issues.apache.org/jira/browse/LOG4J2-3099
Red Hat	JBoss EAP 8.1: Modernizing enterprise Java applications（2025-09-02）	https://developers.redhat.com/blog/2025/09/02/jboss-eap-81-modernizing-enterprise-java-applications
その他	The Twelve-Factor App：XI. Logs	https://12factor.net/logs
社内	Log4j2_EFS_ローリングアップデート_FD競合_完全解説（同じ PC の Log4j_EFS_Rolling。5.5 を本資料で訂正）	C:¥Users¥taka_¥Claude¥Log4j_EFS_Rolling
社内	ECS_EFS_Dockerfile_Symboliclink_lite（いまのリンク構成・mid/ の設計）	C:¥Users¥taka_¥Claude¥ECS_EFS_Dockerfile_Symboliclink_lite